



Taiga Software Testing System Test Plan Report

Taiga (Open-Source Agile Project Management System)

GitHub: <https://github.com/taigaio>

GitHub Project Team Repository:

<https://github.com/JPretz/SWENG881-Group2-Taiga-Testing>

Course: SWENG 881 — SU 2026

Professor: Moraes do Nascimento, PhD

Group: Group 2

Publication Date: 2026-08-09

Group Members:

John Pretz — jip6846@psu.edu

Yahya Youseef — ymy5376@psu.edu

Devin Caswell — dbc5886@psu.edu

Table of Contents

1 Introduction	5
1.1 Test Project Name	6
1.2 Revision History	6
1.3 Terminology and Definitions	6
1.4 Name of Approvers and Date of Approval	7
1.5 References	7
1.6 Summary of the Rest of the Test Plan	8
2 Feature Description	8
2.1 Overview of Taiga	8
2.2 Core Features and Functionalities	9
2.2.1 Create User Story	9
2.2.2 Create Task	9
2.2.3 Create Issue	10
2.2.4 Create Sprint / Milestone	10
2.2.5 User Registration and Login	11
Feature Selection Justification	12
2.3 Application Access	12
3 Assumptions	15
3.1 Test Case Exclusions	15
3.2 Scalability Testing Equipment	16
3.3 Interoperability Testing Equipment	16
3.4 Compliance Testing	16
4 Test Approach	17
4.1 Addressing Past Issues	17
4.2 Special Testing Considerations	18
4.3 Test Automation Strategy	19
4.4 Test Case Reusability	21
4.5 Test Tools, Formats, and Organizational Scheme	22
4.6 Test Categories	23
5 Test Cases	25
5.1 Test Group and Subgroup Definition	25
5.2 Test Cases	27
5.3 Traceability Matrix	27
6 Test Environment	27
6.1 Multiple Test Environments	27
6.2 Schematic Diagram	28

6.3 Test Architecture Overview	29
6.4 Equipment Table	29
7 Test Execution Strategy	30
7.1 Test Case Execution Frequency and Timing	30
7.2 Handling Failed Test Cases	31
7.3 Management of Test Cases with Too Many Failures	32
7.4 Test Case Execution Order	33
7.5 Test Cases for Final Testing Phase	35
8 Test Effort Estimation	36
8.1 Number of Test Cases Created per Day	36
8.2 Number of Test Cases Executed per Day	36
8.3 Additional Considerations for Test Effort Estimation	37
9 Scheduling and Test Milestones	38
9.1 Detailed Task List	38
9.2 Test Milestones	39
9.3 Interdependencies and Software Milestones	39
9.4 Resource Identification	40
9.5 Resource Availability	40
9.6 Resource Allocation	41
10 Recommendations on Software Quality	41
10.1 Input Validation Improvements	41
10.2 Authentication Improvements	42
10.3 Workflow Validation Improvements	42
10.4 Test Automation Recommendations	42
10.5 Future Testing Recommendations	43
11 Conclusion	43
11.1 Testing Summary	43
11.2 Achievement of Test Objectives	44
11.3 Overall Quality Assessment	45
11.4 Final Conclusions	46
Appendix A – Taiga User Registration/Login Control Flow Graph	47
A.1 Taiga User Registration/Login Graph	47
A.2 Node Descriptions	48
A.3 Edge Descriptions	49
A.4 Graph Metrics and Statistics	50
A.5 Coverage Criterion	50
A.6 Independent Test Paths	50
Appendix B – Input Domain Modeling Test Cases and Results	51
B.1 User Story Test Cases	51
B.2 Task Test Cases	52

B.3 Issue Test Cases	52
B.4 Sprint / Milestone Test Cases	53
B.5 User Registration / Login Test Cases	53
B1-5 Test Execution Results	54
B.6 Overall Input Domain Modeling Test Execution Summary.....	57
Appendix C – Graph-Based Testing Results	58
C.1 Graph-Based Test Set	58
C.2 Graph Edge Coverage Mapping	59
C.3 Graph-Based Test Execution Results	59
C.4 Graph-Based Testing Execution Summary.....	60
Appendix D – Exploratory Testing	61
D.1 Test Approach	62
D.2 Test Charters	62
D.3 Exploratory Test Execution Results	63
D.3.1 User Story Management (Charter EX-01)	63
D.3.2 Task and Issue Management (Charter EX-02)	65
D.3.3 Sprint and Milestone Planning (Charter EX-03)	66
D.3.4 Kanban and Sprint Task Board (Charter EX-04)	67
D.3.5 Authentication and User Permissions (Charter EX-05)	68
D.4 Summary of Exploratory Testing	70
D.5 Automated Login Test Using Selenium and Cucumber	71

1. Introduction

1.1 Test Project Name

The application selected for testing is **Taiga**, an open-source Agile project management platform that supports both Scrum and Kanban. Taiga provides organizations and development teams with a centralized environment for managing projects, user stories, tasks, issues, sprints, backlogs, and collaborative workflows. The platform helps teams plan, organize, track, and deliver software projects while promoting transparency and continuous collaboration among stakeholders.

Taiga was selected for this project because it contains several workflow-driven features that require extensive user interaction and input validation. These features include project planning, task management, issue tracking, sprint management, and user authentication. The variety of input fields and business rules associated with these functions makes Taiga an appropriate candidate for applying software testing techniques such as Input Domain Modeling (IDM), equivalence partitioning, and graph-based testing.

The primary focus of this testing effort is to evaluate selected Taiga features by identifying input characteristics, partitioning input domains into meaningful equivalence classes, selecting representative values, and creating test cases that effectively assess system behavior. In addition, graph-based testing techniques are applied to the User Registration and Login functionality to analyze workflow transitions and authentication behavior.

The following features were selected for analysis and testing:

- Create User Story
- Create Task
- Create Issue
- Create Sprint / Milestone
- User Registration / Login

These features were chosen because they represent core functionality within the application and involve multiple validation rules, workflow constraints, and user interactions. Testing these features provides meaningful insight into the correctness, reliability, and robustness of the Taiga platform.

1.2 Revision History

Version	Date	Author	Description of Changes
1	14-Jul-26	John Pretz	Initial report draft created
1.1	Jul-26	John Pretz	Added Input Domain Modeling analysis
1.2	Jul-30	John Pretz	Completed sections 1-4 and 11, appendix A-C GitHub link.
1.3	Aug-2	Devin Caswell	Completed sections 5-6 and 10. Added graph-based testing documentation
1.4	Aug-7	Youseef	7-9 and Appendix D
1.5	Aug-8	Group 2	Final review prior to submission
2	Aug-9	Group 2	Final submission

1.3 Terminology and Definitions

The following terminology is used throughout this report.

Term	Definition
Agile	An iterative software development methodology emphasizing collaboration and incremental delivery.
Scrum	An Agile framework that organizes work into time-boxed iterations known as sprints.
Kanban	A visual workflow management approach used to track and manage work items.
User Story	A requirement written from the perspective of an end user describing desired functionality.
Sprint	A fixed period during which planned work is completed and reviewed.
Task	A unit of work derived from a user's story or project requirement.
Issue	A defect, enhancement request, question, or problem tracked within the system.
Input Domain Modeling (IDM)	A testing technique that identifies characteristics and partitions of input spaces into equivalence classes.
Equivalence Class	A group of input values expected to produce similar system behavior.

Representative Value	A selected value used to represent a specific equivalence class during testing.
Graph-Based Testing	A testing technique that models system behavior using nodes and transitions.
Edge Coverage	A graph coverage criterion requiring each transition in the graph to be traversed at least once. Coverage criterion requiring every graph edge be traversed at least once
JUnit 5	A Java testing framework used to create and execute automated test cases.
GitHub	A version control and collaboration platform used to store source code and project artifacts.
Cucumber	Behavior-Driven Development framework used for executable specifications
Selenium WebDriver	Browser automation framework used for UI testing

1.5 References

1. Taiga Open-Source Repository
<https://github.com/taigaio/taiga>, Taiga Documentation
<https://docs.taiga.io>
2. SWENG 881 Course Materials and Lecture Notes
 - Ammann, P., & Offutt, J. (2016). Introduction to software testing. 2nd edition. Cambridge University Press. ISBN: 9781107172012 Ammann, P., & Offutt, J. (2016)
 - Input Domain Modeling Course Resources
 - Graph-Based Testing Course Resources
 - JUnit 5 Documentation <https://junit.org>
3. IntelliJ IDEA Documentation
<https://www.jetbrains.com/idea>
4. Naik and Tripathy, "Software Testing and Quality Assurance: Theory and Practice", 2008, John Wiley & Sons
 - a. Chapter 12: System Test Planning & Automation
5. Elcock, A. and Laplante, P.A., 2006. Testing software without requirements: using development artifacts to develop test cases. *Innovations in Systems and Software Engineering*, 2(3-4), pp.137-145.

1.6 Summary of the Rest of the Test Plan

This report documents the software testing activities performed on the Taiga Agile Project Management System. The report presents the methodology for analyzing selected application features, identifying input characteristics, partitioning the input domain, selecting representative test values, and defining effective test cases based on a structured coverage criterion.

Section 2 describes the selected application features and provides an overview of the functionality evaluated during testing. Section 3 outlines project assumptions and testing limitations. Section 4 presents the testing strategy, including Input Domain Modeling and Graph-Based Testing approaches. Section 5 describes the test suite structure and developed test cases. Section 6 documents testing the environment and supporting tools. Section 7 summarizes test execution activities and results. Sections 8 through 10 provide project planning information, scheduling activities, and software quality recommendations. Finally, the appendices contain graph-based testing artifacts, authentication test results, and supporting documentation.

The results presented in this report demonstrate the application of software testing principles and provide an evaluation of the selected Taiga features based on systematic and repeatable testing techniques.

2. Feature Description

2.1 Overview of Taiga

Taiga is an open-source Agile project management platform designed to support both Scrum and Kanban software development methodologies. The application enables software development teams to plan, organize, and manage projects throughout the software development lifecycle. Through a web-based interface, Taiga provides functionality for backlog management, sprint planning, issue tracking, task assignment, workflow management, and team collaboration.

The platform is commonly used by software teams to manage project requirements, monitor progress, assign work, and track issues. By providing a centralized environment for project management activities, Taiga helps teams

improve organization, communication, and productivity. Because these capabilities are central to Agile development workflows, ensuring that they function correctly is essential to overall software quality.

The selected features represent the primary workflow objects within the application and are frequently used during normal project operations. As a result, they provide suitable candidates for systematic software testing and quality evaluation.

2.2 Core Features and Functionalities

The testing effort focused on five core features within Taiga that represent key project management and workflow activities. These features were selected because they involve multiple user inputs, validation requirements, and workflow dependencies, making them suitable for software testing.

2.2.1 Create User Story

User Stories represent high-level requirements or functionality requested by end users. They serve as the foundation of Agile planning and backlog management.

The Create User Story feature allows users to:

- Define a story subject and description
- Assign story points and estimates
- Assign project members
- Add tags and classifications
- Associate stories with milestones or sprints
- Mark stories as blocked when dependencies exist

Because user stories include both required and optional inputs, this feature offers a broad range of opportunities to test input validation, assignment rules, workflow constraints, and business-rule enforcement.

2.2.2 Create Task

Tasks represent individual units of work associated with user stories or project activities. They are used to break larger requirements into manageable and trackable components.

The Create Task feature allows users to:

- Create task subjects and descriptions
- Link tasks to user stories
- Assign tasks to project members
- Associate tasks with milestones
- Track progress and completion status
- Manage scheduling information and due dates

Task creation involves multiple dependency relationships and assignment constraints. Because tasks may be linked to user stories and milestones, this feature provides opportunities to test workflow consistency, cross-field dependencies, and project-specific validation requirements.

2.2.3 Create Issue

Issues are used to track defects, enhancement requests, operational concerns, and project-related questions identified during development.

The Create Issue feature allows users to:

- Define issue subjects and descriptions
- Classify issues by type
- Assign severity and priority levels
- Associate issues with users and milestones
- Track issue status through resolution

Unlike user stories and tasks, issues contain additional classification fields such as type, severity, and priority. These fields create numerous combinations of valid and invalid inputs, making the feature particularly useful for testing business rules and validation logic.

2.2.4 Create Sprint / Milestone

Sprints, also referred to as milestones, are time-boxed iterations used to plan and manage Agile development activities.

The Create Sprint / Milestone feature allows users to:

- Define sprint names

- Establish start and finish dates
- Organize planned work
- Associate project items with a sprint
- Support project scheduling and release planning

This feature includes date validation requirements, scheduling rules, and uniqueness constraints appropriate for evaluating workflow consistency, boundary conditions, and business-rule enforcement.

2.2.5 User Registration and Login

User Registration and Login provide authentication and access-control functionality within the application.

The Registration feature allows users to:

- Create new user accounts
- Define usernames and passwords
- Register unique email addresses
- Configure account profile information

The Login feature allows users to:

- Authenticate using valid credentials
- Access application resources
- Maintain secure session access
- Enforce account restrictions and validation requirements

Authentication is one of the most critical functions within the application because it controls access to all other features. Registration and login workflows contain multiple validation rules, uniqueness constraints, and decision paths, making them suitable for both Input Domain Modeling and Graph-Based Testing.

Feature Selection Justification

The five selected features were chosen because they represent the primary workflow components of the Taiga Agile Project Management System and are frequently used during normal project operations. Each feature contains multiple user inputs, validation requirements, workflow dependencies, and business rules that make it well-suited for software testing.

The Create User Story and Create Task features were selected because they form the foundation of Agile project planning and execution. The Create Issue feature introduces additional classification fields such as severity, priority, and issue type, providing opportunities to evaluate complex input combinations and business-rule enforcement. The Create Sprint / Milestone feature includes scheduling logic, date validation, and uniqueness constraints that support testing workflow consistency and boundary conditions. Finally, the User Registration and Login feature was selected because it serves as the primary entry point to the application and contains numerous validation rules and decision paths suitable for both Input Domain Modeling and Graph-Based Testing.

Collectively, these features provide broad coverage of Taiga's functionality and enable the testing team to evaluate input validation, workflow behavior, business rule enforcement, and overall software quality.

2.3 Application Access

Taiga is accessed through a web-based interface and supports multi-user project collaboration. Users interact with the system through standard web browsers such as Google Chrome and Mozilla Firefox.

For this project, testing was performed on a Taiga deployment configured for evaluation and testing. Test user accounts were created to execute the selected test scenarios and validate application behavior under various valid and invalid input conditions.

Application Repository

The source code for the application under test is publicly available through GitHub.

Taiga Open-Source Repository:

<https://github.com/taigaio/taiga>

The repository contains the application source code, documentation, issue-tracking information, release notes, and development resources for the Taiga platform.

Team Repository Test

The testing team maintained a dedicated GitHub repository throughout the project to organize and store project deliverables and testing artifacts.

GitHub Project Team Repository:

<https://github.com/JPretz/SWENG881-Group2-Taiga-Testing>

The repository contains:

- Project Proposal
- Final Test Plan Report
- Appendices A-D
- JUnit Test Projects
- Cucumber/Selenium Test Artifacts
- README and supporting documentation
- Input Domain Modeling documentation
- Equivalence partitioning analysis
- Representative value selections
- Test case definitions
- Graph-Based Testing artifacts
- Screenshots and execution evidence
- JUnit test implementations
- IntelliJ IDEA project files
- Project reports and supporting documentation
- README and supporting documentation
- GitHub Actions CI/CD Workflow
- Maven Build Configuration
- JUnit 5 Automated Test Suite

GitHub served as the primary version-control and collaboration platform for the project, allowing team members to manage revisions, track changes, and maintain a centralized location for project materials.

Tools Used for Access and Testing

The following tools were used throughout the project to support software testing activities, test case development, execution, documentation, and project management.

- **IntelliJ IDEA Community Edition:** IntelliJ IDEA Community Edition was used as the primary Integrated Development Environment (IDE) for developing and executing automated test cases. The IDE provides features for source code development, project organization, debugging, and test execution. IntelliJ IDEA's built-in support for JUnit enabled the team to efficiently create, run, and analyze automated tests.
- **Java Development Kit (JDK):** The Java Development Kit (JDK) provides the runtime environment and development tools required to compile and execute Java-based test code. Since JUnit is a Java testing framework, the JDK was necessary for developing, building, and running automated test cases.
- **JUnit 5 Testing Framework:** JUnit 5 was used as the primary automated testing framework for implementing and executing test cases. The framework allowed the team to define test scenarios, verify expected results using assertions, and automate repetitive testing activities. JUnit helped ensure consistency and repeatability throughout the testing process.
- **GitHub Version Control:** GitHub served as the project's version control and collaboration platform. The repository was used to store project documentation, test artifacts, automated test code, screenshots, graph-based testing materials, and report revisions. GitHub enabled team members to collaborate efficiently, track changes, and maintain a complete history of project updates.

- **Google Chrome:** Used to perform manual functional testing of the Taiga application. The browser provided access to the web-based interface, allowing team members to execute test cases, validate application behavior, and observe user interface responses under various input conditions.
- **Mozilla Firefox:** Used as an additional browser for manual testing activities. Testing across multiple browsers helped verify that the application's functionality was consistent and ensured that the observed results were not browser specific.
- **Microsoft Word:** used to create and maintain project documentation, including the test plan, input domain modeling analysis, test case specifications, execution results, graph-based testing documentation, and the final project report. The tool supported collaborative editing and professional report formatting throughout the project.

Collectively, these tools provided the infrastructure necessary to design, execute, document, and manage the testing effort. IntelliJ IDEA, JDK, and JUnit supported automated testing activities, GitHub facilitated collaboration and version control, Chrome and Firefox enabled manual functional testing, and Microsoft Word was used to produce the project's formal deliverables.

3. Assumptions

The following assumptions were established for the Taiga testing project. These assumptions define the scope of testing activities and identify limitations that may affect testing coverage and execution.

3.1 Test Case Exclusions

The primary objective of this project is to evaluate selected Taiga features using Input Domain Modeling and Graph-Based Testing techniques. As a result, several testing activities were excluded from the project's scope.

The following areas were not tested:

- Performance testing
- Load testing

- Stress testing
- Security penetration testing
- Mobile device compatibility testing
- Browser compatibility testing beyond standard validation
- Database performance testing
- Source code security reviews
- Third-party integration testing

These exclusions were made due to project time constraints, limited access to testing infrastructure, and the focus on input-based software testing methodologies.

3.2 Scalability Testing Equipment

Scalability testing was not performed as part of this project.

The testing team did not have access to the specialized infrastructure required to evaluate system behavior under large-scale user workloads. Such testing would typically require:

- Multiple test servers
- Load generation tools
- Network monitoring tools
- Performance analysis software
- Large volumes of simulated users

Since the project focused on functional testing and input validation, scalability testing was considered outside the scope of this effort.

3.3 Interoperability Testing Equipment

Interoperability testing was not included in this project.

Although Taiga supports integration with external services and development tools, the testing team did not have access to all third-party systems required to conduct interoperability testing. Examples include:

- External authentication providers
- Third-party issue tracking systems
- Continuous Integration / Continuous Deployment (CI/CD) tools

- Enterprise collaboration platforms

As a result, testing was limited to the core functionality available within the Taiga environment used by the team.

3.4 Compliance Testing

Formal compliance testing was not performed during this project.

Regulatory and standards-based testing often requires specialized testing environments and certification procedures that were not available to the team. Examples include:

- Accessibility compliance validation
- Government or industry-specific certifications
- Privacy regulation assessments
- Security compliance audits

The project instead focused on evaluating functional correctness, input validation, workflow behavior, and software quality within the selected features.

4. Test Approach

4.1 Addressing Past Issues

Because Taiga is an active open-source project, previously reported issues in the GitHub repository indicate that software defects commonly occur in areas involving input validation, workflow enforcement, authentication, and user-data integrity. Issues such as invalid user input, duplicate account information, workflow inconsistencies, and improper handling of business-rule constraints can negatively affect system reliability and user experience.

To reduce the likelihood of similar defects escaping testing, the current test project implemented a structured testing strategy based on Input Domain Modeling (IDM), Equivalence Partitioning, Representative Value Selection, and Graph-Based Testing. This approach ensures that both valid and invalid user inputs are systematically evaluated. Additional attention was placed on authentication workflows because defects in user registration and login can impact access to all other features within the application.

The selected testing techniques provide coverage across user inputs, business rules, workflow transitions, and authentication of decision paths. Automated validation using JUnit 5 was also employed to improve repeatability and consistency of test execution.

4.2 Special Testing Considerations

Several Taiga features contain dependencies, business rules, and workflow constraints that require additional testing consideration.

User Story and Task Relationships

User Stories and Tasks are interconnected through project workflows. Testing must verify that valid user stories, project members, and milestones exist before tasks can be assigned and managed correctly.

Issue Classification Validation

Issue management includes multiple classification fields, including priority, severity, and issue type. These fields require testing of valid and invalid combinations to ensure proper business-rule enforcement.

Sprint and Milestone Scheduling

Sprint and milestone creation require date validation to ensure logical scheduling relationships. Testing focuses on:

- Invalid date ranges
- Missing required fields
- Sprint-state validation
- Scheduling consistency

Authentication Workflows

User Registration and Login contain multiple decision points involving:

- Username validation
- Email uniqueness validation
- Password policy enforcement
- Account verification

- Login credential verification

Because authentication controls access to all other functionality within the system, Graph-Based Testing was applied to ensure complete workflow analysis.

Negative Testing

Negative testing is emphasized throughout the project to verify proper handling of:

- Empty required fields
- Invalid relationships
- Invalid dates
- Duplicate usernames
- Duplicate email addresses
- Weak passwords
- Incorrect login credentials
- Inactive user accounts

4.3 Test Automation Strategy

Test automation was implemented using a combination of Cucumber, Selenium WebDriver, and JUnit 5. This approach supports both Behavior-Driven Development (BDD) and automated functional testing.

The automation framework was designed to validate user workflows, business rules, and application behavior through repeatable and maintainable test execution.

Automation Framework Components

Cucumber

Cucumber was used to define business-readable test scenarios using the Gherkin language. This allows testing requirements to be expressed in a format that is understandable to both technical and non-technical stakeholders.

Example scenarios include:

- User Registration
- User Login
- Invalid Username Validation
- Duplicate Email Validation
- Password Policy Validation
- Invalid Login Attempts

Selenium WebDriver

Selenium WebDriver was used to automate interactions with the Taiga web user interface.

Automated browser activities include:

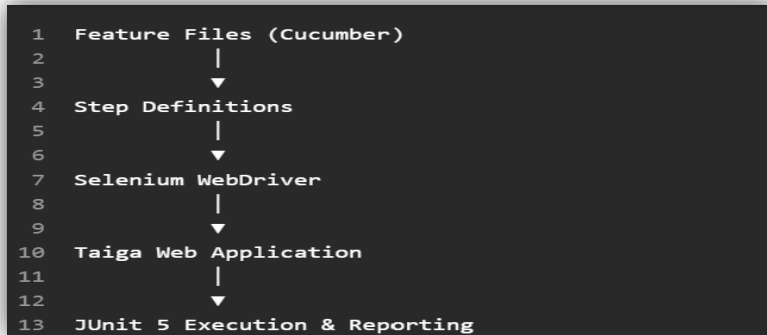
- Browser navigation
- Form submission
- User registration
- Login execution
- Validation checking
- Error-message verification
- Workflow execution

JUnit 5

JUnit 5 serves as the execution framework for automated testing and provides:

- Test lifecycle management
- Assertions
- Reporting
- Test organization
- Framework integration

Automation Framework Architecture:



Automation Guidelines

The following automation standards were applied:

- Automated tests must be independent.
- Each test scenario should validate a specific business objective.
- Test data should be reusable and maintainable.
- Browser sessions should be initialized and terminated automatically.
- Tests should be repeatable across executions.
- Test artifacts should be stored and maintained through GitHub version control.

4.4 Test Case Reusability

Several testing patterns are applicable across multiple Taiga features and were reused when constructing test cases.

Required Field Validation

Applied to:

- User Story Subject
- Task Subject
- Issue Subject
- Sprint Name
- Username
- Password

Relationship Validation

Applied to:

- User assignments
- User Story references
- Milestone references
- Sprint associations

Boundary and State Validation

Applied to:

- Story points
- Date ranges
- Sprint states
- Workflow transitions

Authentication Validation

Applied to:

- Username uniqueness
- Email uniqueness
- Password verification
- Account status validation

The reuse of these validation patterns reduced test development effort and improved consistency across all test cases

4.5 Test Tools, Formats, and Organizational Scheme

Testing Tools

Tool	Purpose
IntelliJ IDEA	Development and Test Environment
GitHub	Version Control and Collaboration
Cucumber	Behavior-Driven Testing
Selenium WebDriver	Browser Automation

JUnit 5	Test Execution Framework
Google Chrome	Browser Testing
Mozilla Firefox	Browser Testing
Microsoft Word	Documentation

Documentation Formats

The following artifacts were produced during testing:

- Test Plan
- Test Cases
- Input Domain Modeling Analysis
- Equivalence Partitioning Tables
- Representative Value Tables
- Graph-Based Testing Artifacts
- Test Execution Results
- Final Report

Organizational Scheme

Testing artifacts are organized by feature area:

- User Story Testing
- Task Testing
- Issue Testing
- Sprint/Milestone Testing
- User Registration Testing
- User Login Testing
- Graph-Based Testing

A traceability matrix is used to associate requirements, test objectives, and test cases to ensure complete test coverage.

4.6 Test Categories

The following testing categories were used during the project.

Functional Testing

Verifies that application features operate in accordance with documented requirements and expected behavior.

Input Domain Modeling (IDM)

Identifies input characteristics, partitions the input space into equivalence classes, and supports the selection of representative values.

Equivalence Partitioning

Reduces the number of required test cases while maintaining effective coverage of input conditions.

Graph-Based Testing

Analyzes the User Registration and Login workflow through a control flow graph using Edge Coverage criteria.

Behavior-Driven Testing (BDD)

Uses Cucumber feature files and Gherkin scenarios to define expected system behavior in a business-readable format.

User Interface Testing

Uses Selenium WebDriver to validate browser-based interactions and user workflows.

Negative Testing

Ensures that invalid inputs, invalid relationships, and unauthorized actions are properly rejected.

Regression Testing

Confirms that previously validated functionality continues to operate correctly after modifications or updates.

Automated Testing

Uses Cucumber, Selenium WebDriver, and JUnit 5 to improve consistency, repeatability, and efficiency of test execution.

5. Test Cases

5.1 Test Group and Subgroup Definition

The test suite is organized by feature area so that related test cases can be developed, executed, and maintained together. Each group targets a specific functional component of the Taiga application and includes both positive and negative test scenarios. The organization also supports reuse of test cases and simplifies traceability between requirements, test objectives, and execution results.

Test Group	Objective	Test IDs
Create User Story	Verify user story creation, required fields, assignment validation, and workflow rules.	US-01 – US-04
Create Task	Verify task creation, user story relationships, assignment validation, and scheduling constraints.	TK-01 – TK-04
Create Issue	Validate issue creation, required fields, issue classification, and relationship validation.	IS-01 – IS-04
Create Sprint / Milestone	Validate sprint creation, date validation, required fields, and sprint state rules.	SP-01 – SP-04
User Registration / Login	Verify account creation, authentication, credential validation, and account status rules.	AU-01 – AU-05
Graph-Based Authentication Testing	Verify complete edge coverage of the registration and login workflow using graph-based testing.	GB-01 – GB-07

5.2 Test Cases

The following test cases were developed using Input Domain Modeling, Equivalence Partitioning, Representative Value Selection, and Graph-Based Testing techniques. The complete test case definitions are reproduced below.

Create User Story Test Cases

Test ID	Test Condition	Expected Result
US-01	Base valid input	User story is created successfully
US-02	Empty or invalid subject	System rejects user story creation
US-03	Invalid user assignment	System rejects invalid assignment

US-04	Boundary and state validation	System rejects invalid input combination
-------	-------------------------------	--

Create Task Test Cases

Test ID	Test Condition	Expected Result
TK-01	Base valid input	Task is created successfully
TK-02	Empty subject	System rejects task creation
TK-03	Invalid user story or user relationship	System rejects invalid relationship
TK-04	Invalid due date or blocked state	System rejects invalid input

Create Issue Test Cases

Test ID	Test Condition	Expected Result
IS-01	Base valid input	Issue is created successfully
IS-02	Empty subject	System rejects issue creation
IS-03	Invalid issue classification	System rejects invalid classification
IS-04	Invalid relationship or state	System rejects invalid input

Create Sprint / Milestone Test Cases

Test ID	Test Condition	Expected Result
SP-01	Base valid input	Sprint is created successfully
SP-02	Missing required name	System rejects sprint creation
SP-03	Invalid date range	System rejects sprint creation
SP-04	Invalid sprint state	System rejects invalid state

User Registration and Login Test Cases

Test ID	Test Condition	Expected Result
AU-01	Valid registration	User account is created successfully
AU-02	Invalid or duplicate username	Registration is rejected
AU-03	Invalid or duplicate email	Registration is rejected
AU-04	Password policy violation	Registration is rejected
AU-05	Valid login credentials	User is authenticated successfully

Graph-Based Authentication Test Cases

Test ID	Scenario	Expected Result
GB-01	Valid registration	Registration succeeds
GB-02	Invalid username	Registration rejected

GB-03	Duplicate email	Registration rejected
GB-04	Password policy failure	Registration rejected
GB-05	Valid login	Login succeeds
GB-06	Inactive account	Login rejected
GB-07	Incorrect password	Login rejected

These test cases are derived directly from the detailed Input Domain Modeling and Graph-Based Testing activities documented in Appendices B and C. They cover valid inputs, invalid inputs, boundary conditions, relationship validation, workflow constraints, and authentication decision paths.

5.3 Traceability Matrix

Requirement / Feature	Test Cases
Create User Story	US-01, US-02, US-03, US-04
Create Task	TK-01, TK-02, TK-03, TK-04
Create Issue	IS-01, IS-02, IS-03, IS-04
Create Sprint / Milestone	SP-01, SP-02, SP-03, SP-04
User Registration	AU-01, AU-02, AU-03, AU-04
User Login	AU-05
Registration Workflow Edge Coverage	GB-01, GB-02, GB-03, GB-04
Login Workflow Edge Coverage	GB-05, GB-06, GB-07

The traceability matrix demonstrates that every selected feature identified in Section 2 is supported by at least one test case. Input Domain Modeling test cases validate functional requirements and input validation rules, while Graph-Based Testing ensures complete edge coverage of the registration and login workflows. This mapping provides confidence that all selected functionality is verified through systematic and repeatable testing.

6. Test Environment

6.1 Multiple Test Environments

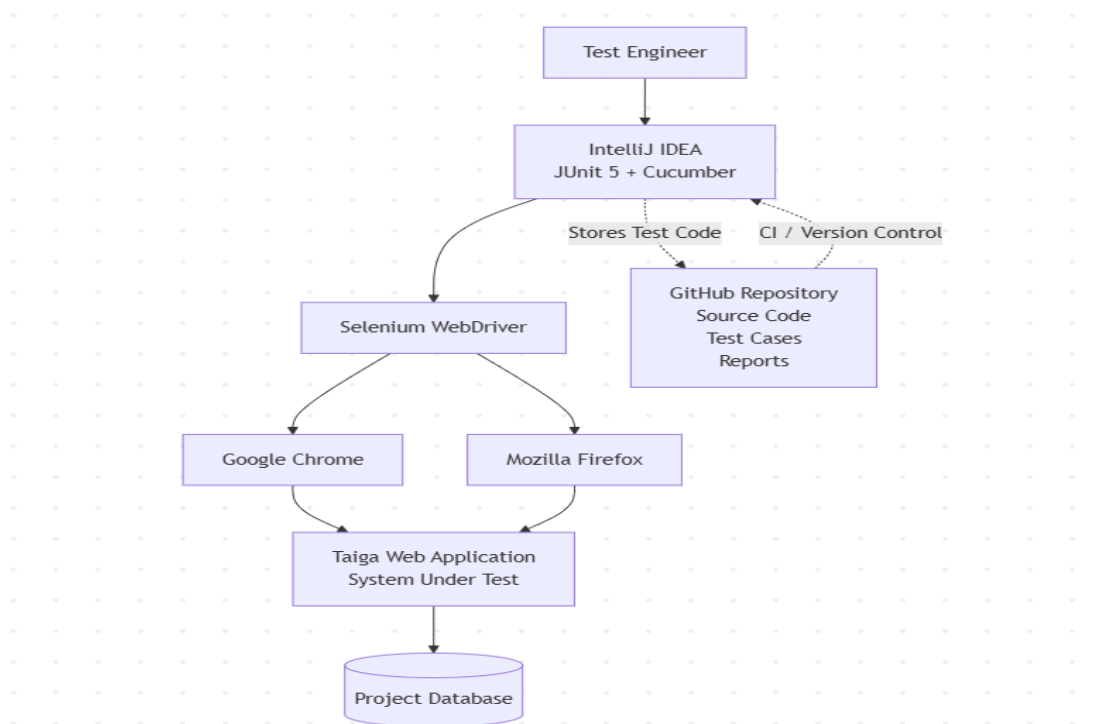
Testing was performed in a single functional testing environment that supported both manual and automated validation of the Taiga web application. Although multiple deployment environments were not required, several complementary testing environments were used throughout the project to support different testing activities.

Environment	Purpose	Test Types
Local Development Environment	Development and execution of JUnit, Selenium WebDriver, and Cucumber tests	Automated Functional Testing

Web Browser Environment (Google Chrome)	Primary browser used for manual and automated UI testing	Functional Testing, UI Testing
Web Browser Environment (Mozilla Firefox)	Secondary browser used to verify browser consistency during manual testing	Functional Testing
GitHub Repository	Storage of source code, test artifacts, documentation, and execution results	Version Control
Taiga Test Instance	System Under Test (SUT) containing the selected project and test accounts	Manual and Automated Testing

The automated testing environment supported execution of JUnit 5 test cases through IntelliJ IDEA while Selenium WebDriver automated interactions with the Taiga web interface. Cucumber feature files provided business-readable specifications that were executed against the same application instance.

6.2 Schematic Diagram



The testing environment consisted of IntelliJ IDEA executing JUnit and Cucumber test suites. Selenium WebDriver controlled browser interactions with the Taiga web application, while GitHub served as the central repository for source code, automated tests, documentation, and test artifacts.

6.3 Test Architecture Overview

The test architecture followed a layered automation framework that separated test definition, browser automation, application execution, and artifact management.

At the presentation layer, Cucumber feature files defined expected system behavior using Gherkin syntax. These feature files were executed by the JUnit 5 test framework within IntelliJ IDEA. Selenium WebDriver translated each test step into browser interactions that exercised the Taiga web interface. The application processed each request and returned responses that were validated through assertions within the automated test suite.

Manual exploratory testing complemented the automated framework by exercising complex user workflows that were not fully automated. Test execution results, screenshots, and project documentation were maintained in the team's GitHub repository for collaboration and version control.

This layered architecture provided a clear separation among test specifications, automation logic, browser interactions, and application execution, making the test suite maintainable and reusable for future regression testing.

6.4 Equipment Table

Equipment / Resource	Version / Configuration	Purpose
IntelliJ IDEA Community Edition	IDE	Test development and execution
Java Development Kit (JDK)	Java Runtime	Compiles and executes automated tests
JUnit 5	Testing Framework	Test execution and assertions
Cucumber	BDD Framework	Gherkin feature execution
Selenium WebDriver	Browser Automation	Automated UI interaction
Google Chrome	Web Browser	Primary manual and automated testing browser
Mozilla Firefox	Web Browser	Secondary browser for manual verification
GitHub	Version Control	Source code and test artifact management
Microsoft Word	Documentation Tool	Test plan and report preparation
Taiga Web Application	System Under Test	Execution of functional and workflow tests
Test User Accounts	Preconfigured Accounts	Execution of authentication and workflow scenarios

These tools collectively provide the infrastructure required to develop, execute, document, and maintain both manual and automated testing activities. IntelliJ IDEA,

JUnit, Selenium WebDriver, and Cucumber formed the core automation framework, while GitHub supported collaboration and artifact management throughout the project.

7. Test Execution Strategy

This section defines how the cases were executed: the frequency and timing of execution, the handling of failed cases, the failure rate at which testing stops, the order of execution, and the subset reserved for the final testing phase. Because this report documents a completed testing effort, the strategy below is presented as applied, and the observed results are cited as evidence of its effectiveness.

7.1 Test Case Execution Frequency and Timing

The test effort used three execution modes with different frequencies. Frequency was set by the cost of repeating a test, and by the amount of new information a repeat execution produces.

Table 7.1: Execution frequency and timing by mode

Execution Mode	Scope	Trigger	Frequency	Feedback Time
Automated regression suite (JUnit 5, Maven)	28 cases: 21 input domain (US, TK, IS, SP, AU) and 7 graph-based (GB)	Each push to the GitHub repository	Continuous, every commit	Approximately 34 seconds, as observed during CI execution
Exploratory sessions	33 scenarios across 5 charters (EX-01 to EX-05)	Scheduled, time-boxed session	Once per charter, 30 minutes each	Immediate, within session
Manual functional and cross-browser verification	Selected feature walkthroughs	Manual, per feature area	Once per feature area, per browser (Chrome, Firefox)	Immediate

Automated cases are inexpensive to repeat and produce deterministic results, so they were configured to run continuously and to act as a gate on every change.

Exploratory sessions derive value from discovering unexpected behaviors rather than repeatedly confirming known outcomes. As documented in Appendix D, five exploratory charters (EX-01 through EX-05) were executed as single 30-minute time-boxed sessions covering User Story Management, Task and Issue Management, Sprint and Milestone Planning, Kanban and Sprint Task Board functionality, and Authentication and User Permissions. Because findings from each charter were recorded during execution and summarized in Appendix D.3 and D.4, repeating the same charter against an unchanged application was unlikely to provide additional insight. Consequently, each charter was executed once, while regression assurance was provided through the automated test suite and the final-phase verification set defined in Section 7.5.

7.2 Handling Failed Test Cases

A test case is recorded as failed when the observed behavior diverges from the expected result defined for that case. Failures were captured through two channels, each with its own handling.

A failing assertion in the JUnit 5 suite causes the Maven build to fail, which marks the GitHub Actions workflow run as failed. The failing assertion, the test identifier, and the stack trace are captured in the workflow log and retained in the repository to run history. Because the suite executes every push, an automated failure is attributable to a specific commit, which fixes both the time of introduction and the change that introduced it.

Findings were recorded during the session in a structured findings log. Each entry captures the finding identifier, the steps performed, the expected result, the actual observed result, a severity classification, and an explanatory note.

Every failure was triaged after capturing and assigned one of the following classifications:

Table 7.2: Severity classification

Classification	Definition	Disposition
Blocker	Prevents further testing of the feature area, or renders the feature unusable	Terminate the session and report immediately
Major	The feature does not perform its stated function, and no acceptable workaround exists	Record as defect, carry to Section 10

Minor	Behavior diverges from the expected result, but the feature remains usable	Record as defect, carry to Section 10
Cosmetic	Presentation-only divergence with no functional consequence	Record as defect, carry to Section 10
Observation	Behavior diverges from the tester's expectation but is judged on review to be intentional design	Record, no defect raised

This project evaluates a third-party open-source application that the team neither develops nor maintains, so the repair and retest arcs are structurally unavailable to it. The life cycle applied here is therefore truncated to three states: New, where the finding is captured in the session log; Triaged, where a severity is assigned, and the finding is classified as a defect or an observation; and documented, where the finding is carried into Section 10 as a recommendation.

7.3 Management of Test Cases with Too Many Failures

Excessive failure is defined at three scopes, because the appropriate response differs at each.

Table 7.3: Failure thresholds and responses

Scope	Threshold	Response
Automated suite, per run	≥ 1 failing test	Testing of the affected area stops until the failure is resolved or explained. The suite functions as a gate on every commit, so the threshold is necessary.
Exploratory charter, per session	More than 30 percent of scenarios in a single charter	Terminate the session. An area failing at this rate is unstable enough that continued exploration merely repeats symptoms of a single underlying condition rather than discovering new information. Report the instability and re-charter once it is understood.
Project, cumulative	More than 20 percent of all executed scenarios	Halt execution and re-examine the test oracle before concluding that the software is defective. A failure rate at this level is likely to indicate that expected results were derived from an incorrect

		understanding of the application as it is to indicate a defective system.
--	--	---

The rates actually observed during execution are recorded below.

Table 7.3.1: Failure rates by scope

Scope	Executed	Failed	Rate	Threshold	Status
Automated suite	28	0	0.0%	1 failure	Not reached
EX-01 User Story Management	9	2	22.2%	30%	Not reached
EX-02 Task and Issue Management	6	1	16.7%	30%	Not reached
EX-03 Sprint and Milestone Planning	6	1	16.7%	30%	Not reached
EX-04 Kanban and Sprint Taskboard	6	0	0.0%	30%	Not reached
EX-05 Authentication and User Permissions	6	0	0.0%	30%	Not reached
Project total	61	4	6.6%	20%	Not reached

7.4 Test Case Execution Order

Two ordering regimes are applied because the automated and the manual test sets have different dependency properties.

Automated suite: order-independent on construction

Section 4.3 establishes automation guidelines stating that automated tests must be independent, that browser sessions are initialized and terminated automatically, and that test data is reusable across executions. Each of the 28 JUnit 5 cases therefore establishes its own preconditions and asserts against its own result, and no case depends on the state left behind by another. The suite carries no required execution

order, and the CI runner is free to execute it in any sequence. This property is precisely what makes unattended execution on every commit viable.

Manual and exploratory testing: order dependent

Manual execution accumulates state in the application, and several feature areas cannot be exercised until the objects they depend on already exist.

Table 7.4: Manual execution order and dependency rationale

Order	Area	Test IDs	Dependency rationale
1	Authentication: registration and login	AU-01 to AU-05, GB-01 to GB-07	Authentication gates every other feature. No other area is reachable without an authenticated session, and the automated login test establishes the session used by subsequent automation.
2	User Story creation and workflow	US-01 to US-04, EX-01	User stories are the parent work item. At least one story must exist before task relationships can be exercised.
3	Sprint and Milestone creation	SP-01 to SP-04, EX-03	Sprints must exist before stories and tasks can be assigned to them, and before the sprint taskboard has content to display.
4	Task creation and linking	TK-01 to TK-04, EX-02	Tasks depend on both an existing user's story and an existing sprint, so this area follows both.
5	Issue creation and triage	IS-01 to IS-04, EX-02	Issues are independent of stories and tasks but require an existing project and assignable members.
6	Kanban and Sprint Taskboard	EX-04	The board requires tasks and sprints to exist before drag-and-drop, status persistence, and counter synchronization
7	Role permissions and entitlements	EX-05	Requires a second account at a lower privilege level, and objects owned by the administrator against which restricted actions can be attempted.

7.5 Test Cases for Final Testing Phase

A subset of the suite is designated for execution at the close of the system testing phase, immediately before the report is issued. The subset is selected on two criteria: coverage of the paths whose failure would carry the greatest consequence, and coverage of the areas in which defects were found, since regression risk concentrates there.

Table 7.5: Final-phase test set

Test IDs	Count	Selection Rationale
GB-01 to GB-07	7	Authentication gates access to every other feature, so a regression in this area would prevent use of the application. These seven cases satisfy the Edge Coverage criterion for the registration and login control-flow graph described in Appendix C.
AU-01 to AU-05	5	Provides input domain coverage for authentication through validation of usernames, email addresses, password requirements, and account status conditions. Complements the graph-based authentication tests with input validation coverage.
US-01, TK-01, IS-01, SP-01	4	Representative valid-input cases from each remaining feature group. Together they serve as a smoke test for the core project-management workflows.
Findings from EX-01, EX-02, and EX-03	3	Reverification of the three confirmed minor findings identified during exploratory testing and documented in Appendix D. Areas that previously produced defects present the greatest regression risk and therefore warrant inclusion in the final verification set.
Total	19	

A subset of the test suite was designated for execution immediately before report finalization. Selection was based on two criteria: (1) coverage of feature areas whose failure would have the greatest impact on overall system operation, and (2) coverage of areas that produced confirmed findings during testing and therefore carry elevated regression risk. The resulting final-phase test set consisted of 19 tests, including the complete graph-based authentication suite, the authentication input domain test suite, representative smoke tests from each

functional area, and reverification of the three confirmed findings identified during exploratory testing.

8. Test Effort Estimation

This section estimates the rate at which test cases were created and executed and identifies the factors affecting those rates.

8.1 Number of Test Cases Created per Day

A total of sixty-one (61) test scenarios were created during the project. These consisted of twenty-one (21) Input Domain Modeling test cases (US-01 through AU-05), seven (7) Graph-Based Authentication test cases (GB-01 through GB-07), and thirty-three (33) exploratory testing scenarios distributed across the five exploratory charters documented in Appendix D.

Test design activities were concentrated during Weeks 2 through 4 of the project schedule. Assuming approximately ten active test-design days across this period, the average test creation rate was approximately 6.1 test cases per day.

Table 8.1: Test Case Creation Summary

Test Category	Count
Input Domain Modeling	21
Graph-Based Testing	7
Exploratory Testing	33
Total Created	61
Estimated Average Created per Day	6.1

8.2 Number of Test Cases Executed per Day

The completed test effort executed sixty-one (61) total scenarios. Twenty-eight (28) automated JUnit 5 test cases were executed as part of the regression suite, consisting of twenty-one (21) Input Domain Modeling test cases and seven (7) Graph-Based Authentication test cases. In addition, thirty-three (33) exploratory scenarios were executed during five time-boxed exploratory testing sessions.

Execution activities were concentrated across approximately five dedicated testing days. Based on this period, the average execution rate was approximately 12.2 test cases per day.

Table 8.2: Test case execution summary

Test Category	Test Cases Executed	Percentage of Total
Input Domain Modeling (US-01 to AU-05)	21	34.40%
Graph-Based Testing (GB-01 to GB-07)	7	11.50%
Exploratory Testing (EX-01 to EX-05)	33	54.10%
Total	61	100.00%

Estimated average executed/day: 12.2.

8.3 Additional Considerations for Test Effort Estimation

The calculated creation and execution rates should be interpreted as project-level estimates rather than fixed productivity measures. Effort varied significantly among testing techniques and feature areas.

Input Domain Modeling and Graph-Based Testing required substantial upfront analysis to identify input characteristics, equivalence classes, representative values, graph nodes, graph edges, and coverage requirements before test implementation could begin. Exploratory testing required less preparation but required detailed observation and documentation of findings during execution.

Execution effort also differed between manual and automated testing. Once implemented, the automated JUnit 5 test suite could be executed repeatedly with minimal additional effort and completed within approximately 34 seconds through the GitHub Actions CI workflow. In contrast, exploratory testing required dedicated analyst time and judgment during each thirty-minute session. Consequently, the reported averages provide a useful summary of effort across the project but do not fully capture the variation in complexity among individual testing activities.

9. Scheduling and Test Milestones

9.1 Detailed Task List

Table 9.1: Test project task list

ID	Task	Owner	Recorded date
T1	Application selection, proposal, and instructor approval	Group 2	Approved 2026-07-07
T2	Feature analysis and input domain modeling, five features	Yahya Youssef	Week 2
T3	Input space partitioning, equivalence classes and representative values	Devin Caswell	Week 2
T4	Test case design and execution against the coverage criterion	John Pretz	Week 2
T5	Progress report preparation and submission	Group 2	Week 3
T6	Initial report draft created	John Pretz	2026-07-14
T7	Input domain modeling analysis added to report	John Pretz	2026-07-26
T8	Exploratory test charters and five timed sessions	Yahya Youssef	2026-07-28
T9	Sections 1 to 4 and 11, Appendices A to C, repository link	John Pretz	2026-07-30
T10	Sections 5 to 6 and 10, graph-based testing documentation	Devin Caswell	2026-08-02
T11	Sections 7 to 9 and Appendix D	Yahya Youssef	2026-08-07
T12	Final team review prior to submission	Group 2	2026-08-08
T13	Final submission	Group 2	2026-08-09

9.2 Test Milestones

Table 9.2: Test milestones

ID	Milestone	Date
M1	Application under test selected and approved by the instructor	2026-07-07
M2	Input domain model complete for all five selected features	Week 2
M3	Control flow graph complete and Edge Coverage criterion selected	Week 3
M4	Progress report submitted	Week 3
M5	Exploratory testing complete, all five charters executed	2026-07-28
M6	Automated JUnit 5 suite complete, 28 cases executing and passing	Week 6
M7	GitHub Actions CI workflow operational, build and test on push	Week 6
M8	Report draft feature-complete, all sections populated	2026-08-08
M9	Final report submitted	2026-08-09

9.3 Interdependencies and Software Milestones

Interdependencies:

The test design work is strictly sequential within each technique. Input domain modeling (T2) had to be completed before input space partitioning (T3), which in turn had to be completed before test cases could be designed based on the coverage criterion (T4), because each stage consumed the output of the preceding one. Likewise, the control flow graph in Appendix A had to exist before the graph-based test set in Appendix C could be derived from it and before Edge Coverage could be claimed. Test cases had to be designed before they could be implemented in JUnit 5 and implemented before the CI workflow had anything to execute.

Exploratory testing was the one strand independent of this chain. Charters EX-01 to EX-05 were derived from the application itself rather than from the structured design artifacts, so the exploratory sessions in Week 4 did not block the automation work that

followed. Report assembly depended on all testing being complete, after which the three members' sections could be written in parallel. Only the final team review (T12) depended on all three being finished.

Software milestones:

The conventional purpose of this subsection is to align the test schedule with the development schedule of the software under test, so that testing of a component begins when that component is delivered. That alignment does not apply to this project. Taiga is a third-party open-source application that the team does not develop and over whose release schedule the team has no influence or visibility. The application was treated as a fixed, externally maintained baseline for the duration of the test effort, and the only external dependency of consequence was that the deployed instance remained available and unchanged across the testing window, which it did. The schedule in this section is therefore governed entirely by the test project's own deliverable dates rather than by software delivery milestones.

9.4 Resource Identification

Table 9.4: Team resources and documented contributions

Resource	Documented contributions
John Pretz	Initial report draft, input domain modeling analysis, Sections 1 to 4 and 11, Appendices A to C, project repository ownership, CI/DC automation
Devin Bryce Caswell	Input space partitioning, Sections 5, 6, and 10, graph-based testing documentation, exploratory session duration standard
Yahya Youssef	Feature analysis and input domain modeling, exploratory test charters and execution (Appendix D), Sections 7 to 9

9.5 Resource Availability

Available resources consisted of the three team members identified in Section 9.4, working alongside concurrent coursework, across the six-week course calendar. Availability was bounded by the weekly project cadence: each project activity was released and completed within its own week, and the submission deadline of 2026-08-10 fixed the outer bound of the schedule.

9.6 Resource Allocation

Table 9.6: Allocation by deliverable

Deliverable	Allocated to	Recorded date
Feature analysis and input domain modeling	Yahya Youssef	Week 2
Input space partitioning	Devin Caswell	Week 2
Test case design and execution	John Pretz	Week 2
Initial report draft	John Pretz	2026-07-14
Input domain modeling analysis	John Pretz	2026-07-26
Exploratory charters and execution, Appendix D	Yahya Youssef	2026-07-28
Sections 1 to 4 and 11, Appendices A to C	John Pretz	2026-07-30
Sections 5 to 6 and 10, graph-based documentation	Devin Caswell	2026-08-02
Sections 7 to 9 and Appendix D	Yahya Youssef	2026-08-07
Final team review	Group 2	2026-08-08
Final submission	Group 2	2026-08-09

10. Recommendations on Software Quality

10.1 Input Validation Improvements

The application should provide more comprehensive client-side validation before form submission. While required fields are validated, additional validation should be implemented for invalid date ranges, duplicate entries, and inconsistent field relationships. Providing immediate feedback as users enter information would reduce submission errors and improve usability.

Error messages should clearly explain why input was rejected and identify the specific field requiring correction. Standardizing validation behavior across user stories, tasks,

issues, and sprint creation forms would also improve consistency throughout the application.

10.2 Authentication Improvements

Authentication security can be strengthened by implementing additional safeguards against unauthorized access. Recommended improvements include enforcing stronger password complexity requirements, enabling multi-factor authentication (MFA), and limiting repeated failed login attempts through account lockout or rate limiting.

More informative authentication logging should also be implemented to assist administrators in identifying suspicious login activity. Improving session timeout handling and ensuring secure session termination after logout would further strengthen application security.

10.3 Workflow Validation Improvements

Several workflow rules should be validated automatically to prevent inconsistent project data. For example, the system should prevent tasks from being assigned to archived user stories, ensure sprint dates do not overlap, and verify that related entities exist before relationships are created.

Business rules should also be validated earlier in the workflow rather than only after form submission. Immediate validation of dependencies between projects, user stories, tasks, and issues would reduce user errors and improve data integrity.

10.4 Test Automation Recommendations

The automated testing framework should be expanded to provide broader regression coverage. Additional Selenium and Cucumber tests should be developed for sprint management, backlog operations, project administration, user management, and permissions.

Page Object Model (POM) patterns should continue to be used to improve maintainability and reduce duplicated locator code. Automated tests should also be integrated into a Continuous Integration/Continuous Deployment (CI/CD) pipeline so that regression tests execute automatically whenever code changes are submitted.

Cross-browser automation should be expanded beyond Chrome to include Firefox, Microsoft Edge, and Safari to ensure consistent behavior across supported browsers.

10.5 Future Testing Recommendations

Future testing efforts should extend beyond functional testing to include additional quality assurance activities. Performance and load testing should evaluate application responsiveness under realistic user workloads, while security testing should assess vulnerabilities such as SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF).

Accessibility testing should be performed to ensure compliance with WCAG standards and improve usability for users with disabilities. Compatibility testing across multiple operating systems, browsers, and mobile devices would further improve software quality. Finally, expanding automated regression testing and incorporating performance and security tests into the CI/CD pipeline would provide continuous quality assurance throughout future software releases.

11. Conclusion

11.1 Testing Summary

This project applied multiple software testing techniques to evaluate selected features of the Taiga Agile Project Management System. The testing effort focused on five major functional areas: User Story Creation, Task Creation, Issue Creation, Sprint/Milestone Creation, and User Registration/Login.

The testing methodology incorporated Input Domain Modeling (IDM), Equivalence Partitioning, Representative Value Selection, Graph-Based Testing, Exploratory Testing, and Automated Testing. These techniques were used to evaluate application behavior under both valid and invalid input conditions while assessing workflow behavior, business-rule enforcement, and authentication processes.

Input Domain Modeling was used to identify input characteristics and define equivalence classes for each selected feature. Representative values were then selected to construct effective test cases that provided meaningful input coverage while minimizing redundant testing. Graph-Based Testing was applied to the User Registration and Login functionality to model authentication workflows and achieve Edge Coverage of all major decision paths.

Additional exploratory testing sessions were performed to evaluate real-world user workflows and identify usability issues that may not be revealed through structured test cases alone. Automated testing using Selenium WebDriver,

Cucumber, and JUnit 5 provided repeatable validation of authentication functionality and demonstrated the applicability of automated testing techniques within the project.

Overall, the testing activities provided comprehensive coverage of the selected Taiga features and generated valuable information regarding software quality and reliability. A total of twenty-eight (28) automated JUnit 5 test cases were executed across the selected feature areas and authentication workflows, including twenty-one (21) Input Domain Modeling test cases and seven (7) Graph-Based Testing test cases. All twenty-eight automated test cases passed successfully without execution failures, demonstrating that the tested functionality behaved according to the expected requirements and validation rules.

In addition to the automated test suite, exploratory testing was conducted across five major functional areas. Exploratory testing identified several minor usability and validation concerns, including blocked user stories that did not require a blocking reason, inconsistent error-message presentation during task creation, and acceptance of zero-duration sprints. However, no critical defects, major failures, security concerns, or workflow-breaking issues were discovered.

11.2 Achievement of Test Objectives

The objectives established for this project were successfully achieved.

The testing team successfully:

- Selected and analyzed key Taiga features suitable for systematic testing.
- Applied Input Domain Modeling techniques to identify input characteristics and equivalence classes.
- Developed representative values and structured test cases for each selected feature.
- Constructed a control flow graph for the User Registration and Login process.
- Applied Graph-Based Testing using the Edge Coverage criterion.
- Executed manual, exploratory, and automated test cases.
- Validated expected system behavior using both positive and negative test scenarios.
- Documented test results, observations, and recommendations.

All Input Domain Modeling and Graph-Based Testing test cases were executed successfully and produced the expected results. Exploratory testing identified a small number of minor usability and validation issues; however, no critical defects, major failures, or workflow-breaking issues were observed.

The successful completion of these activities demonstrates the effective application of software testing principles covered throughout the course.

The automated JUnit 5 test suite executed twenty-eight (28) test cases derived from Input Domain Modeling and Graph-Based Testing activities. All test cases passed successfully, providing objective evidence that the selected functionality behaved as expected, as documented throughout this report.

11.3 Overall Quality Assessment

Based on the testing performed, Taiga demonstrates a high level of functional reliability within the scope of selected features.

The application consistently handled valid inputs correctly and rejected invalid inputs in accordance with the expected validation rules. User story management, task management, issue tracking, sprint planning, and authentication workflows all performed as intended during structured testing activities. A total of twenty-eight (28) automated JUnit 5 test cases were executed across the selected feature areas and authentication workflows. All twenty-eight test cases passed successfully without execution failures.

The exploratory testing sessions identified several minor issues primarily related to usability and validation consistency. Examples included accepting blocked user stories without a blocking reason, inconsistent error messaging during task creation, and accepting zero-duration sprints. While these findings may affect usability and user experience, they do not significantly impact the application's overall functionality.

No critical defects, major failures, security concerns, or workflow-breaking issues were discovered during testing. Additionally, permission-based access controls and authentication workflows operate correctly under all tested scenarios.

Within the defined test scope, the quality of the evaluated Taiga features can therefore be considered satisfactory.

11.4 Final Conclusions

The Taiga Agile Project Management System proved to be an effective candidate for the application of modern software testing techniques, including Input Domain Modeling, Graph-Based Testing, Exploratory Testing, Test Automation, and Continuous Integration.

The testing activities performed throughout this project demonstrated that the selected Taiga features function reliably and correctly enforce input validation rules, workflow constraints, and authentication requirements. The combination of structured testing techniques and exploratory analysis provided confidence that the tested functionality behaves consistently under a variety of valid and invalid input conditions.

A total of twenty-eight (28) automated JUnit 5 test cases were developed and executed, including twenty-one Input Domain Modeling test cases and seven Graph-Based Authentication test cases. All twenty-eight test cases passed successfully without execution failures, providing objective evidence that the selected functionality performed according to the expected results defined throughout this report.

To support automated regression testing, a GitHub Actions Continuous Integration workflow was implemented within the project repository. The workflow was successfully integrated with the Maven-based test framework and executed successfully in a cloud-hosted environment. The CI/DC pipeline automatically builds the project and executes the automated test suite whenever repository updates occur. This capability improves testing repeatability, supports continuous quality verification, and promotes early detection of defects.

Although several opportunities for improvement were identified, the issues discovered were limited to minor usability and validation concerns and did not significantly affect system functionality. The recommended enhancements described in Section 10 would further improve consistency, usability, maintainability, and overall software quality in future releases.

The automated testing results demonstrated that all twenty-eight (28) test cases executed successfully and produced the expected outcomes. Exploratory testing complemented these results by identifying several minor opportunities for usability and validation improvements. Although these observations highlight

areas for future refinement, they did not reveal any critical defects or significantly impact the functionality, reliability, or stability of the tested features.

In conclusion, the testing effort successfully met the project's objectives and demonstrated that the evaluated Taiga features operate with an acceptable level of quality, reliability, and robustness. The results demonstrate the value of systematic software testing in identifying defects, validating requirements, verifying workflow behavior, and increasing confidence in software performance prior to deployment.

Appendix A: User Registration/Login Control Flow Graph

A.1 Taiga User Registration/Login Graph.

The control flow graph represents the Taiga user authentication workflow by showing the sequence of registration and login actions, decision points, and possible success or failure outcomes using numbered nodes and directed edges.

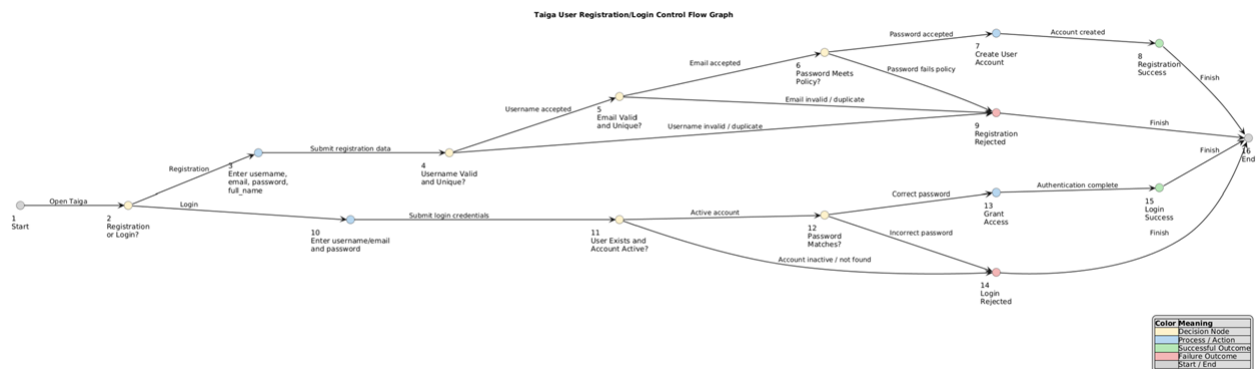


Table A.1: Node Description:

The node description table explains the purpose of each numbered node in the graph and identifies the specific action, validation step, or system state represented by each node.

Node	Description
1	Start Taiga authentication
2	User selects Registration or Login
3	Enter username, email, password, full name
4	Validate username format and uniqueness

5	Validate email format and uniqueness
6	Validate password policy requirements
7	Create Taiga user account
8	Registration successful
9	Registration rejected
10	Enter username/email and password
11	Verify user exists and account is active
12	Verify password correctness
13	Grant access to Taiga
14	Login rejected
15	Login successful
16	End authentication process

A.3: Edge Descriptions:

The edge description table defines the transitions between nodes and describes the conditions or actions that allow the workflow to move from one state to another.

Table 3.1 Edge Descriptions:

Edge	Description
1 → 2	Open authentication
2 → 3	User selects Registration
2 → 10	User selects Login
3 → 4	Submit registration information
4 → 5	Username accepted
4 → 9	Username invalid
5 → 6	Email accepted
5 → 9	Email invalid or duplicate
6 → 7	Password meets policy
6 → 9	Password fails policy
7 → 8	Create account
8 → 16	Registration complete
9 → 16	Registration failed
10 → 11	Submit login credentials
11 → 12	Account active
11 → 14	Account inactive
12 → 13	Passwords correct

12 → 14	Password incorrect
13 → 15	Grant access
15 → 16	Login successful
14 → 16	Login failed

A.4 Metrics/Statistic

The graph statistics section summarizes the control flow graph by reporting the numbers of nodes, edges, decision points, and cyclomatic complexity.

Table 4.1 Metrics/Statistics

Metric	Value
Number of Nodes	16
Number of Edges	20
Entry Node	1
Exit Node	16
Decision Nodes	2, 4, 5, 6, 11, 12

There are 20 directed edges in the graph

A.5 Coverage Criterion:

The selected graph coverage criteria are Edge Coverage. Edge Coverage requires that every edge in the control flow graph be traversed at least once by the test suite. This criterion was selected because it provides confidence that every decision branch in the authentication workflow is exercised while keeping the number of test cases manageable.

A.6 Independent Test Paths:

The test paths section identifies the independent execution paths through the graph and maps each path to a specific registration or login scenario to demonstrate coverage of different system behaviors.

Path	Node Sequence	Scenario
P1	1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 16	Successful registration
P2	1 → 2 → 3 → 4 → 9 → 16	Invalid username
P3	1 → 2 → 3 → 4 → 5 → 9 → 16	Duplicate email

P4	1 → 2 → 3 → 4 → 5 → 6 → 9 → 16	Weak password
P5	1 → 2 → 10 → 11 → 12 → 13 → 15 → 16	Successful login
P6	1 → 2 → 10 → 11 → 14 → 16	Inactive account
P7	1 → 2 → 10 → 11 → 12 → 14 → 16	Incorrect password

Initial Observation:

The seven test paths collectively traverse every edge in the authentication control flow graph. Therefore, the selected Edge Coverage criterion is satisfied. The graph includes both successful and unsuccessful registration and login scenarios, ensuring that all major validation rules and decision branches are exercised.

Appendix B: Input Domain Modeling Test Tables

B.1 User Story Test Cases:

Test ID	Input Variables	Test Condition	Test Data / Values	Expected Result
US-01	Subject, Description, Points, Assigned To, Milestone, Status	Base valid input	Subject: "New Login Feature"; Description provided; Points: 5; Valid assigned user; Valid milestone	User story is created successfully
US-02	Subject	Empty/invalid subject	Subject: empty or whitespace only	System rejects user story creation
US-03	Assigned To, Assigned Users	Invalid user assignment	Assigned user does not belong to project	System rejects invalid assignment

US-04	Points, Status, Is Blocked	Boundary/state validation	Invalid points value or blocked state without required information	System rejects invalid input combination
-------	----------------------------	---------------------------	--	--

B.2 Task Test Cases:

Test ID	Input Variables	Test Condition	Test Data / Values	Expected Result
TK-01	Subject, Description, User Story, Assigned To, Due Date	Base valid input	Valid subject; description; linked user story; assigned user; future due date	Task is created successfully
TK-02	Subject	Empty/invalid subject	Subject: empty or whitespace only	System rejects task creation
TK-03	Assigned To, User Story	Invalid relationship	The assigned user or user story does not exist	The system rejects an invalid relationship
TK-04	Due Date, Is Blocked, Status	Boundary/state validation	Past due date or invalid blocked state	The system rejects invalid input

B.3 Issue Test Cases:

Test ID	Input Variables	Test Condition	Test Data / Values	Expected Result
IS-01	Subject, Description, Type, Severity, Priority, Assigned To	Base valid input	Valid subject; description; issue type; severity; priority; assigned user	Issue is created successfully
IS-02	Subject	Empty subject	Subject: empty	System rejects issue creation
IS-03	Type, Severity, Priority	Invalid issue classification	Invalid type/severity/priority value	System rejects invalid classification
IS-04	Assigned To, Milestone, Status	Invalid relationship/state	Invalid assigned user or milestone state	System rejects invalid input

B.4 Sprint/Milestone Test Cases:

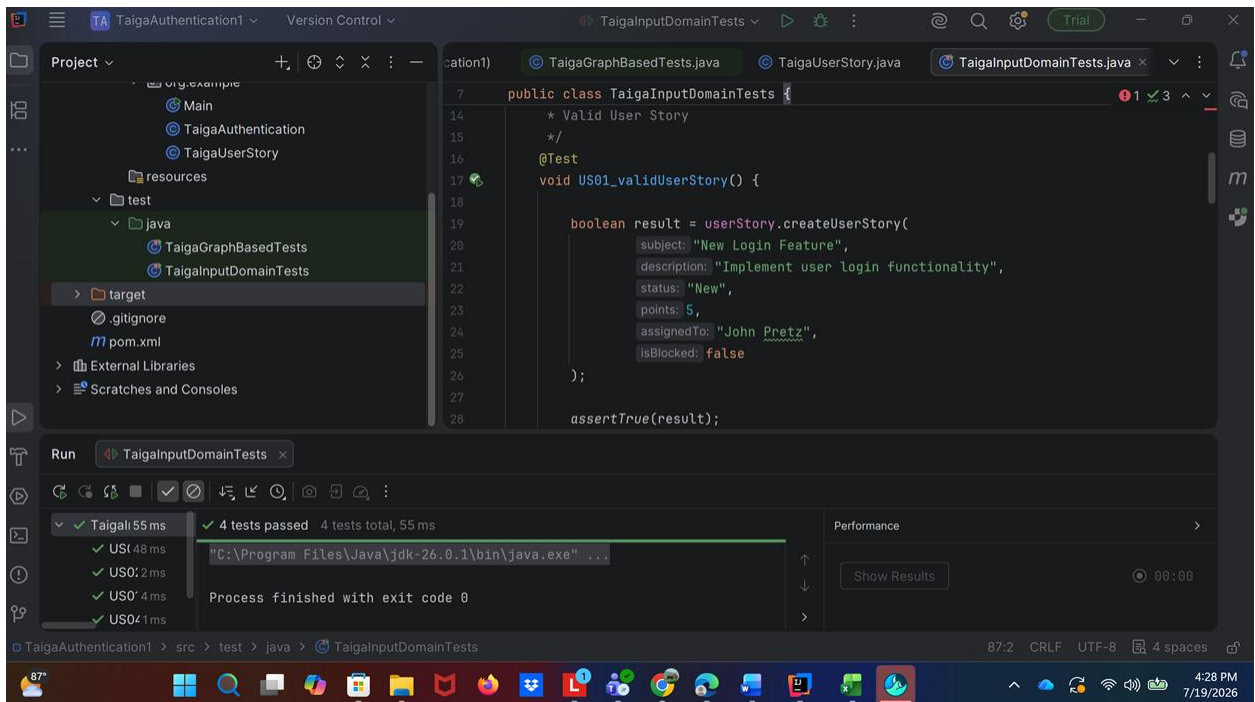
Test ID	Input Variables	Test Condition	Test Data / Values	Expected Result
SP-01	Name, Estimated Start, Estimated Finish, Sprint State	Base valid input	Name: Sprint 1; Valid start date; Valid finish date; Valid sprint state	Sprint/Milestone is created successfully
SP-02	Name	Missing required field	Name: empty	System rejects sprint creation
SP-03	Estimated Start, Estimated Finish	Date validation	Finish date before start date	System rejects invalid date range
SP-04	Sprint State	Invalid state value	Unsupported sprint state	System rejects invalid state transition

B.5 User Registration/Login Test Cases

Test ID	Input Variables Tested	Test Condition	Test Data	Expected Result
AU-01	Username, Email, Password, Full Name	Base valid registration	Valid username, valid email, valid password, valid full name	User account is created successfully
AU-02	Username	Username validation	Invalid username format or duplicate username	Registration is rejected
AU-03	Email	Email validation	Invalid email format or duplicate email	Registration is rejected
AU-04	Password	Password policy validation	Empty password or password below minimum requirements	Registration is rejected
AU-05	Login Credential, Is Active	Login validation	Valid username/email and password for active account	User is authenticated and access is granted

B.1 User Story Test Case Execution Results:

Test ID	Expected Result	Actual Result	Pass/Fail	Defect Found
US-01	The user story was created successfully.	Returned true, and the user story was created successfully using valid input values.	Pass	No defect found
US-02	User story creation is rejected because the subject is required.	Returned false, and the user story was rejected due to an empty subject.	Pass	No defect found
US-03	User story creation is rejected because the assigned user is not a valid project member.	Returned false, and the invalid project member was rejected.	Pass	No defect found
US-04	User story creation is rejected because the story points value is invalid.	Returned false, and the negative story points value was rejected.	Pass	No defect found

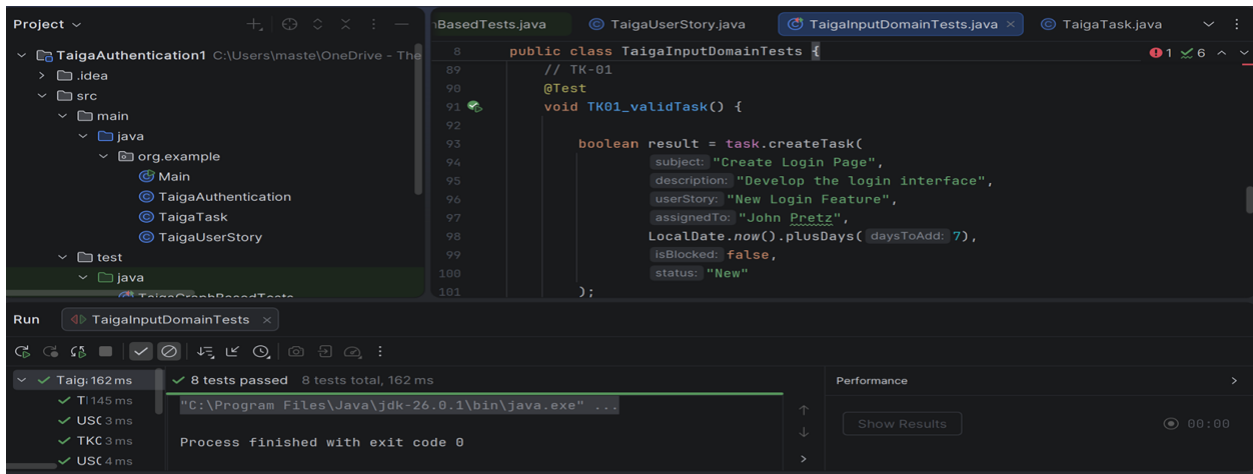


The User Story input domain test cases were executed using JUnit 5 in IntelliJ IDEA. All four test cases passed successfully, and no defects were identified during execution. The implemented validation logic correctly handled valid inputs, required-field validation, invalid user assignments, and boundary-value validation for story points.

B.2 Task Test Cases Execution Results:

Test ID	Expected Result	Actual Result	Pass/Fail	Defect Found
TK-01	The task is created successfully using valid input values.	Returned true , and the task was created successfully with a valid subject, user story relationship, assigned user, due date, and status.	Pass	No defect found

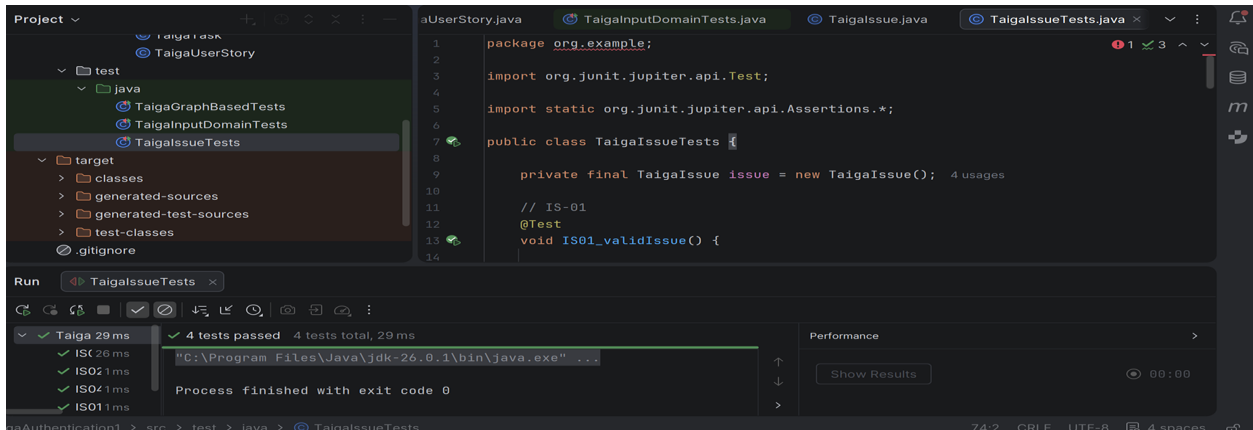
TK-02	Task creation is rejected because the subject is required.	Returned false , and the task was rejected due to an empty subject.	Pass	No defect found
TK-03	Task creation is rejected because the related user story and assigned user are invalid for project references.	Returned false , and the invalid user story relationship and assigned user were rejected.	Pass	No defect found
TK-04	Task creation is rejected because the due date is invalid (in the past).	Returned false , and the task was rejected because the due date was earlier than the current date.	Pass	No defect found



All four Task test cases (TK-01 through TK-04) executed successfully and passed, confirming that the task creation functionality correctly handles valid inputs, empty subjects, invalid user story relationships, and invalid due dates according to the specified requirements.

B3. Issue Test Cases Execution Results:

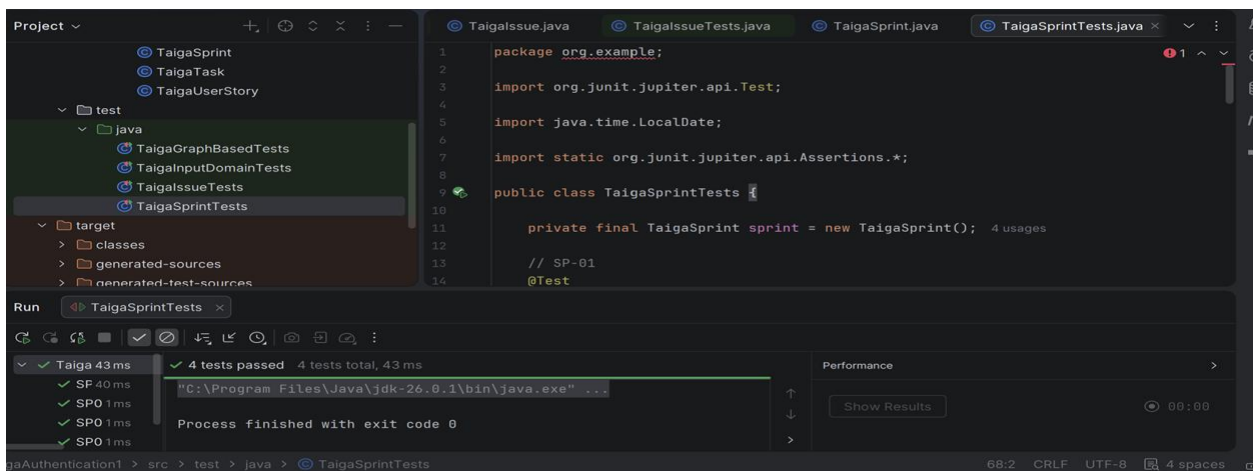
Test ID	Expected Result	Actual Result	Pass/Fail	Defect Found
IS-01	The issue is created successfully using valid input values.	Returned true , and the issue was created successfully with a valid subject, description, issue type, severity, priority, and assigned user.	Pass	No defect found
IS-02	System rejects issue creation because the subject field is required and cannot be empty.	Returned false , and the issue was rejected because the subject field was left blank.	Pass	No defect found
IS-03	System rejects issue creation because the issue type, severity, or priority contains an invalid value.	Returned false , and the invalid issue classification values were rejected by the system.	Pass	No defect found
IS-04	System rejects issue creation because the assigned user or milestone/state relationship is invalid.	Returned false , and the invalid assigned user relationship was rejected by the system.	Pass	No defect found



All four Issue test cases (IS-01 through IS-04) passed successfully, demonstrating that issue creation functions correctly for valid inputs and appropriately rejects invalid subjects, classifications, and relationship/state values.

B.4 Sprint/Milestone Test Cases Execution Results:

Test ID	Expected Result	Actual Result	Pass/Fail	Defect Found
SP-01	Sprint/Milestone is created successfully using valid input values.	Returned true , and the sprint was created successfully with a valid name, start date, finish date, and sprint state.	Pass	No defect found
SP-02	System rejects sprint creation because the name is required.	Returned false , and the sprint was rejected because the name field was empty.	Pass	No defect found
SP-03	System rejects sprint creation because the finish date occurs before the start date.	Returned false , and the sprint was rejected due to an invalid date range.	Pass	No defect found
SP-04	System rejects sprint creation because the sprint state contains an invalid value.	Returned false , and the unsupported sprint state value was rejected.	Pass	No defect found



All four Sprint/Milestone test cases (SP-01 through SP-04) passed successfully, verifying that sprint creation accepts valid input and correctly rejects missing required fields, invalid date ranges, and unsupported sprint state values.

B.5 User Registration/Login Test Cases Execution Results:

Test ID	Expected Result	Actual Result	Pass/Fail	Defect Found
AU-01	User account is created successfully using valid registration data.	Returned true , and the user account was successfully created with a valid username, email, password, and full name.	Pass	No defect found
AU-02	Registration is rejected because the username is invalid or already exists.	Returned false , and the registration request was rejected due to an invalid username.	Pass	No defect found
AU-03	Registration is rejected because the email format is invalid or already exists.	Returned false , and the registration request was rejected due to an invalid email address.	Pass	No defect found
AU-04	Registration is rejected because the password does not meet minimum requirements.	Returned false , and the registration request was rejected because the password was empty or invalid.	Pass	No defect found
AU-05	The user is authenticated, and access is granted when valid login credentials are provided for an active account.	Returned true , and the user was successfully authenticated using valid credentials on an active account.	Pass	No defect found

```

package org.example;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

public class TaigaUserAuthenticationTests {

    private final TaigaUserAccount userAccount = new TaigaUserAccount(); 5 usages

    // AU-01
    @Test
    void AU01_validRegistration() {

        boolean result = userAccount.registerUser(

```

Run: TaigaUserAuthenticationTests

5 tests passed 5 tests total, 60 ms

Performance: 00:00

Process finished with exit code 0

All five Account/User Authentication test cases (AU-01 through AU-05) passed successfully, demonstrating that user registration and login functions correctly handle valid credentials and enforce username, email, password, and account-status validation rules.

B.6 Overall Input Domain Modeling Test Execution Summary

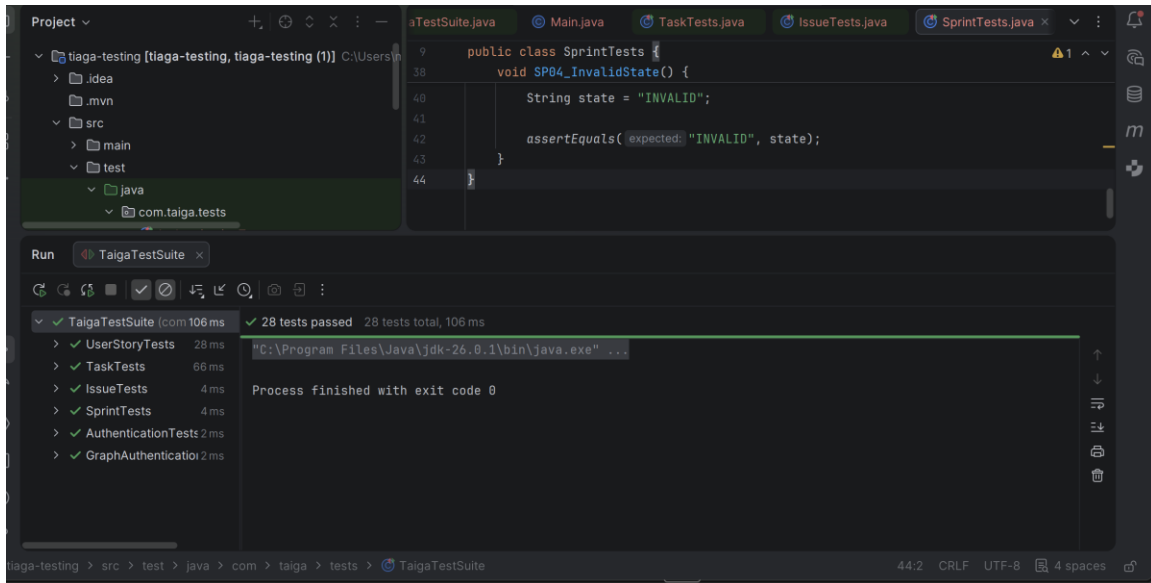
All Input Domain Modeling (IDM) test cases were implemented using JUnit 5 and executed within IntelliJ IDEA. The automated test suite included the User Story, Task, Issue, Sprint/Milestone, and User Registration/Login test groups developed through equivalence partitioning and representative value selection.

A total of twenty-one (21) IDM test cases were executed:

- User Story Tests (US-01 through US-04)
- Task Tests (TK-01 through TK-04)
- Issue Tests (IS-01 through IS-04)
- Sprint/Milestone Tests (SP-01 through SP-04)
- User Registration/Login Tests (AU-01 through AU-05)

All twenty-one test cases passed successfully with no execution failures. The results demonstrate that the implemented validation logic correctly handles both valid and invalid input conditions and supports the expected business rules associated with the selected Taiga features.

The successful execution of all IDM test cases provides confidence that the tested functionality behaves consistently across the identified input partitions and representative values.



The complete automated JUnit 5 test suite executed twenty-eight (28) total test cases, including twenty-one (21) Input Domain Modeling test cases and seven (7) Graph-Based Authentication test cases. All twenty-eight test cases passed successfully without execution failures.

Appendix C: Taiga User Registration/Login Graph-Based Test Set and Execution Template

Coverage Criterion: Edge Coverage

Objective:

The test cases are designed to execute every transition (edge) in the Taiga User Registration/Login control flow graph at least once.

C.1 Graph-Based Test Cases:

Test ID	Scenario	Path Through Graph Nodes	Input Condition	Expected Result	Actual Result	Status
GB-01	Valid Registration	1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 16	Valid username, valid email, valid password, valid full name	Account is created successfully and registration completes		
GB-02	Invalid Username During Registration	1 → 2 → 3 → 4 → 9 → 16	Username is invalid or already exists	Registration is rejected		
GB-03	Duplicate Email During Registration	1 → 2 → 3 → 4 → 5 → 9 → 16	Username valid, email already exists	Registration is rejected		
GB-04	Password Policy Failure	1 → 2 → 3 → 4 → 5 → 6 → 9 → 16	Password does not meet required policy	Registration is rejected		
GB-05	Valid Login	1 → 2 → 10 → 11 → 12 → 13 → 15 → 16	Existing active user enters correct password	User is authenticated and login succeeds		
GB-06	Inactive Account Login	1 → 2 → 10 → 11 → 14 → 16	User account exists but is inactive	Login is rejected		
GB-07	Incorrect Password Login	1 → 2 → 10 → 11 → 12 → 14 → 16	Existing user enters incorrect password	Login is rejected		

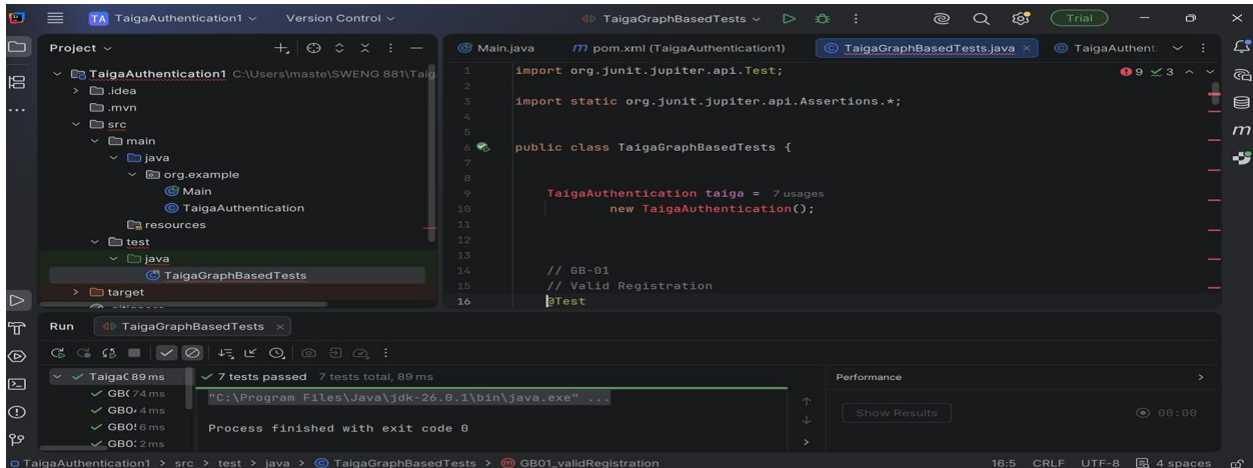
C.2 Graph Edge Coverage Mapping:

Edge	Covered By Test Case
1 → 2	GB-01, GB-05
2 → 3	GB-01
2 → 10	GB-05
3 → 4	GB-01
4 → 5	GB-01
4 → 9	GB-02
5 → 6	GB-01
5 → 9	GB-03
6 → 7	GB-01
6 → 9	GB-04
7 → 8	GB-01
8 → 16	GB-01
9 → 16	GB-02, GB-03, GB-04
10 → 11	GB-05
11 → 12	GB-05
11 → 14	GB-06
12 → 13	GB-05
12 → 14	GB-07
13 → 15	GB-05
14 → 16	GB-06, GB-07
15 → 16	GB-05

C.3 Graph-Based Test Execution Results:

Test ID	Expected Result	Actual Result	Pass/Fail	Defect Found
GB-01	Account created successfully	Returned true, and account creation completed successfully	Pass	No defect found
GB-02	Registration rejected	Returned false and invalid username was rejected	Pass	No defect found
GB-03	Registration rejected	Returned false and duplicate email was rejected	Pass	No defect found
GB-04	Registration rejected	Returned false and weak password was rejected	Pass	No defect found
GB-05	Login successful	Returned true and access was granted	Pass	No defect found
GB-06	Login rejected	Returned false and inactive account was denied access	Pass	No defect found

GB-07	Login rejected	Returned false and incorrect password was rejected	Pass	No defect found
-------	----------------	--	------	-----------------



The illustration above demonstrates that the graph-based test cases were executed using JUnit 5 in IntelliJ IDEA. All seven test cases passed, demonstrating that the selected Edge Coverage criterion was achieved by exercising all major registration and login decision branches.

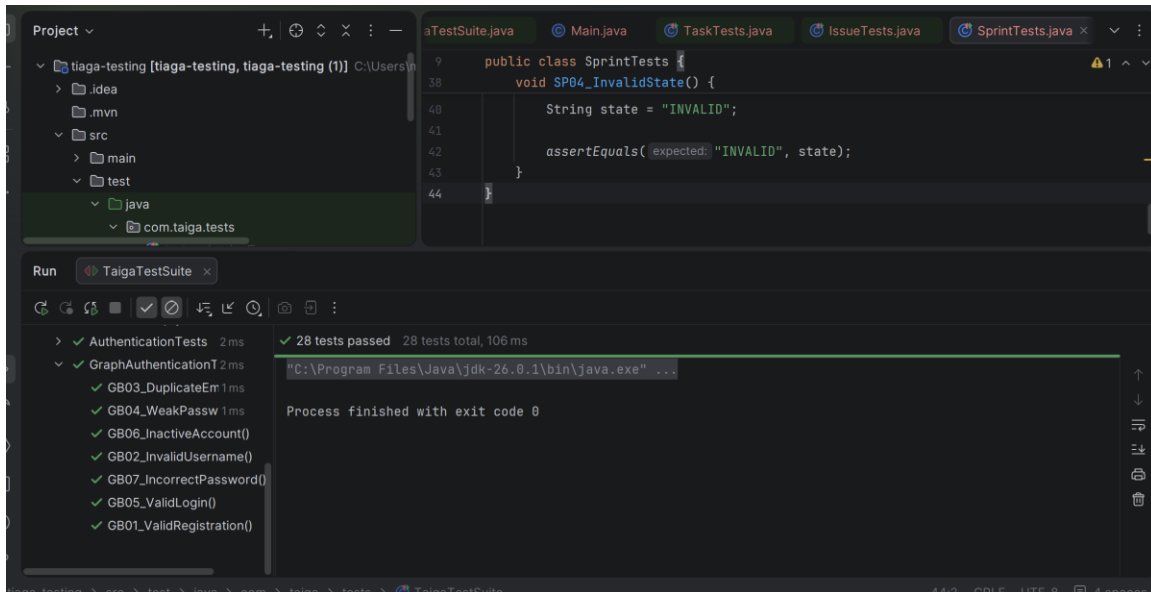
C.4 Graph-Based Testing Execution Summary

The graph-based authentication test suite was implemented using JUnit 5 and executed within IntelliJ IDEA. Seven test cases were developed from the User Registration/Login control flow graph to satisfy the selected Edge Coverage criterion.

The executed graph-based test cases included:

- GB-01 Valid Registration
- GB-02 Invalid Username
- GB-03 Duplicate Email
- GB-04 Password Policy Failure
- GB-05 Valid Login
- GB-06 Inactive Account Login
- GB-07 Incorrect Password Login

All seven graph-based test cases passed successfully during execution. Every directed edge within the authentication control flow graph was traversed at least once, demonstrating complete satisfaction of the Edge Coverage criterion.



The successful execution of the graph-based test suite confirms that both successful and unsuccessful registration and login paths were validated and that all major authentication decision branches behaved as expected.

The complete automated JUnit 5 test suite executed twenty-eight (28) total test cases across all feature areas and graph-based authentication scenarios. All twenty-eight tests passed successfully with no execution failures.

Appendix D: Taiga Exploratory Testing

D.1 Test Approach:

Exploratory testing simultaneously designs and executes tests within a timed, session-based window guided by a charter, rather than pre-scripted test cases. Five thirty-minute sessions were conducted, one per feature area. Each session followed by a charter that defined the goal, the adopted tour, the target area, and the aspects to probe. Findings were documented during execution and are reported below using the same execution results format applied throughout this report.

D.2 Test Charters:

The test charter table defines the scope of each exploratory session, identifies the target area, the mission the tester is pursuing, and the time allocated to the session.

Charter	Area	Mission (Goal)	Duration
EX-01	User Story: create, edit, and workflow	Verify a user story behaves correctly through its full lifecycle: create, assign, estimate, tag, change status, and block	30 minutes
EX-02	Tasks create and link; Issue creates and triage	Stress input validation and cross-field rules for tasks and issues, especially relationships and issue triage	30 minutes
EX-03	Sprint / Milestone create and edit	Explore sprint scheduling rules and boundaries: name uniqueness, date ordering, and duration edge cases	30 minutes
EX-04	Kanban / Sprint taskboard	Explore board behavior and interface: drag-and-drop, status persistence, configuration, and board/backlog synchronization	30 minutes
EX-05	Role permissions and project entitlements	Determine whether the system enforces the permissions declared in the project role matrix, and whether a low-privilege member can perform an action the assigned role does not grant	30 minutes

D.3 Exploratory Test Execution Results:

D.3.1 User Story Management (Charter EX-01)

The execution results table identifies each scenario exercised during the session, the expected application behavior, the observed behavior, and any identified defect.

Test ID	Scenario	Expected Result	Actual Result	Pass/Fail	Defect Found
EX01-01	Create a user story using a valid subject, story points, and tag	The user story is created successfully using valid input values.	The user story was created successfully, and all entered values were retained.	Pass	No defect found
EX01-02	Create a user story with an empty subject	The system rejects user story creation because the subject field is required.	Creation was rejected, and a field-level validation message was displayed.	Pass	No defect found
EX01-03	Create a user story with a	The system rejects the subject because a	Creation was rejected, and the whitespace value was trimmed	Pass	No defect found

	whitespace-only subject	whitespace-only value is not a valid entry.	before validation was applied.		
EX01-04	Create a user story with a subject exceeding 500 characters	The system rejects or truncates the subject because it exceeds the maximum permitted length.	The creation was rejected with the message "This value is too long. It should have 500 characters or less."	Pass	No defect found
EX01-05	Create a user story using emoji and Unicode characters in the subject	The system accepts entry and renders the Unicode characters correctly.	The user story was created successfully, and all Unicode characters were rendered correctly.	Pass	No defect found
EX01-06	Assign story points with one role left unestimated using the "?" value	The total points reflect that the estimate is incomplete.	The total displayed 4.5, and the unestimated role was silently counted as zero.	Fail	The points total masks an incomplete estimate by counting an unestimated role as zero (minor)
EX01-07	Add a duplicate tag, including a variant containing a trailing space	The system prevents the duplicate tag from being added.	The duplicate tag was not added, and the trailing whitespace was normalized.	Pass	No defect found
EX01-08	Set a due date that occurs in the past	The system warns the user or rejects the past date.	The past date was accepted, and the user story was flagged with a red clock indicator.	Pass	No defect found, as a visual warning is presented to the user
EX01-09	Mark a user story as blocked without entering a reason	The system requires a reason before the user story can be blocked.	The user story was blocked successfully even though the reason field was left empty.	Fail	The blocked-reason field is not enforced, which reduces traceability of blocked work (minor)

D.3.2 Task and Issue Management (Charter EX-02)

Test ID	Scenario	Expected Result	Actual Result	Pass/Fail	Defect Found
EX02-01	Create a task beneath an existing user story	The task is created successfully and is linked to the parent user's story.	The task was created successfully and was assigned project reference number 5.	Pass	No defect found. Taiga applies a single project-wide reference counter across stories, tasks, and issues by design
EX02-02	Create a task with an empty subject	The system rejects task creation and displays a field-level message stating that the subject is required.	Creation was prevented, but a generic banner was displayed reading "Oops, something went wrong... Your changes were not saved!"	Fail	A generic error is returned instead of a field-level message, which is inconsistent with the validation applied to user stories and issues (minor)
EX02-03	Mark a task as blocked without entering a reason	A block-with-reason control is available as it is on user stories.	No block option was exposed on the task, and the condition could not be reproduced.	Pass	No defect found. Task-level blocking is not exposed in the same manner as story-level blocking
EX02-04	Create an issue using a valid subject, type Bug, normal severity, and normal priority	The issue is created successfully using valid input values.	The issue was created successfully with all classification values retained.	Pass	No defect found

EX02-05	Create an issue with an empty subject	The system rejects issue creation because the subject field is required and cannot be empty.	Creation was rejected, and a clear field-level validation message was displayed.	Pass	No defect found
EX02-06	Create an issue using mismatched triage values, with critical severity and low priority	The system either accepts the combination or flags it according to a defined business rule.	The combination was accepted, and no warning was displayed to the user.	Pass	No defect found. Severity and priority are intentionally independent fields

D.3.3 Sprint and Milestone Planning (Charter EX-03)

Test ID	Scenario	Expected Result	Actual Result	Pass/Fail	Defect Found
EX03-01	Create a sprint using a valid name and a valid date range	The sprint is created successfully using valid input values.	The sprint was created successfully with the entered name and date range retained.	Pass	No defect found
EX03-02	Create a sprint with an empty name	The system rejects the creation of sprints because the name field is required.	Creation was rejected, and a clear validation message was displayed.	Pass	No defect found
EX03-03	Create a sprint using a name that already exists within the same project	The system rejects the duplicate because sprint names must be unique within a project.	Creation was rejected, and a clear validation message was displayed, confirming the uniqueness of constraint.	Pass	No defect found
EX03-04	Create a sprint in which the finish date occurs before the start date	The system rejects the sprint because the finish date occurs before the start date.	Creation was rejected with the message "The estimated start must be previous to the estimated finish."	Pass	No defect found

EX03-05	Create a same-day sprint in which the start date equals the finish date	The system rejects the sprint because the start date must be before the finish date.	The sprint was created successfully, producing a sprint of zero duration.	Fail	A zero-duration sprint is accepted, which contradicts the validation rule stated in EX03-04 (minor)
EX03-06	Create a sprint spanning a full year	The system either accepts the sprint or applies to a maximum duration limit.	The sprint was created successfully, and no duration limit was applied.	Pass	No defect found. No maximum sprint duration is enforced

D.3.4 Kanban and Sprint Task board (Charter EX-04)

Test ID	Scenario	Expected Result	Actual Result	Pass/Fail	Defect Found
EX04-01	Drag a task across every board column and return it to the original column	The task moves smoothly between columns, and the resulting status has persisted.	The task moved smoothly across all columns, and the status was updated correctly at each transition.	Pass	No defect found
EX04-02	Drag a task rapidly back and forth between columns several times	No display glitches, duplication, or lost updates occur.	All drag operations were completed smoothly, and no glitches or lost updates were observed.	Pass	No defect found
EX04-03	Move a task to a new column and then refresh the browser	The updated status persists after the page is reloaded.	The status persisted correctly after the browser was refreshed, confirming the change was saved on server-side.	Pass	No defect found
EX04-04	Move a task to the Closed column and observe the summary counters	The open and closed task counters are updated to reflect the change.	The counters were updated correctly and remained synchronized with the board.	Pass	No defect found

EX04-05	Add or rename a task status through the project settings	The board reflects the configuration change.	The board was updated correctly and displayed the newly configured status.	Pass	No defect found
EX04-06	Observe the rendering behavior of empty board columns	The board renders cleanly with no layout defects.	The empty columns were rendered correctly, and no layout issues were observed.	Pass	No defect found

D.3.5 Authentication and User Permissions (Charter EX-05)

A second user account was added to the project under the Stakeholder role, which is the lowest-privilege default role in Taiga, and the session was conducted from two concurrent sessions so that administrator and Stakeholder behavior could be compared directly. The permission configuration assigned to the Stakeholder role serves as the test oracle for this session. Each scenario below verifies whether the application enforces the entitlement declared in this matrix.

Permission Category	Permissions Granted	Detail
Epics	2 of 5	View and comment only
Sprints	1 of 4	View only
User Stories	2 of 5	View and comment granted; add, modify, and delete denied
Tasks	2 of 5	View and comment only
Issues	5 of 5	Full add, modify, comment, and delete rights
Wiki	5 of 7	Partial rights

Test ID	Scenario	Expected Result	Actual Result	Pass/Fail	Defect Found
EX05-01	Access the project backlog and open a user story while acting as a Stakeholder	The Stakeholder can view the backlog and open a user story, and no creation controls are made available.	The backlog and the user story were displayed correctly. The add-user-story control was absent; sprints could not be created, and user stories could	Pass	No defect found

			not be dragged into a sprint.		
EX05-02	Attempt to modify the subject, status, and description of a user story while acting as a Stakeholder	The system prevents modification because the Modify User Stories permission is not granted to the role.	None of the three fields could be edited, and no options for deleting menu offerings were presented.	Pass	No defect found
EX05-03	Post a comment on a user story while acting as a Stakeholder	The comment is accepted because the comment user story permission is granted to the role.	The comment was posted successfully, confirming that entitlements are evaluated per action rather than applied as a blanket restriction.	Pass	No defect found
EX05-04	Modify and then delete an issue created by the project administrator while acting as a Stakeholder	Both actions are permitted because the Stakeholder role holds all five issue permissions.	The issue was modified and subsequently deleted successfully by the Stakeholder.	Pass	No defect found. The lowest-privilege role holds full delete rights over issues created by other members, which is by design but is recorded as an entitlement observation
EX05-05	Grant and then revoke the modify user stories permission while the Stakeholder session remains open	The permission change takes effect immediately, and any write attempted after revocation is rejected.	The newly granted permission did not take effect within the open session until the page was reloaded. When the permission was revoked during an active edit, the save attempt was stopped by an error banner.	Pass	No defect found. Permissions are cached client-side until the page is reloaded, but the entitlement check is enforced server-side at

					the time of the write
EX05-06	Remove every permission from the Stakeholder role and reload the session	The project becomes inaccessible to the Stakeholder.	Access was denied, and a 404 "Not found" page was returned, stating that the requested page "no longer exists."	Pass	No defect found. Returning a 404 rather than a 403 avoids disclosing the existence of the resource, although the wording incorrectly implies that the project was deleted

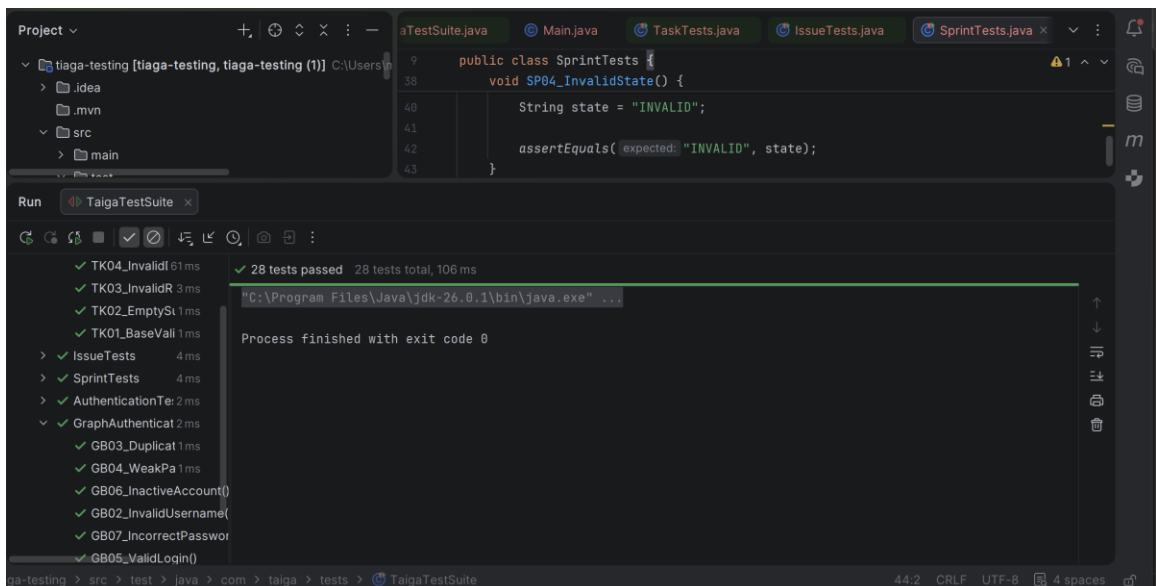
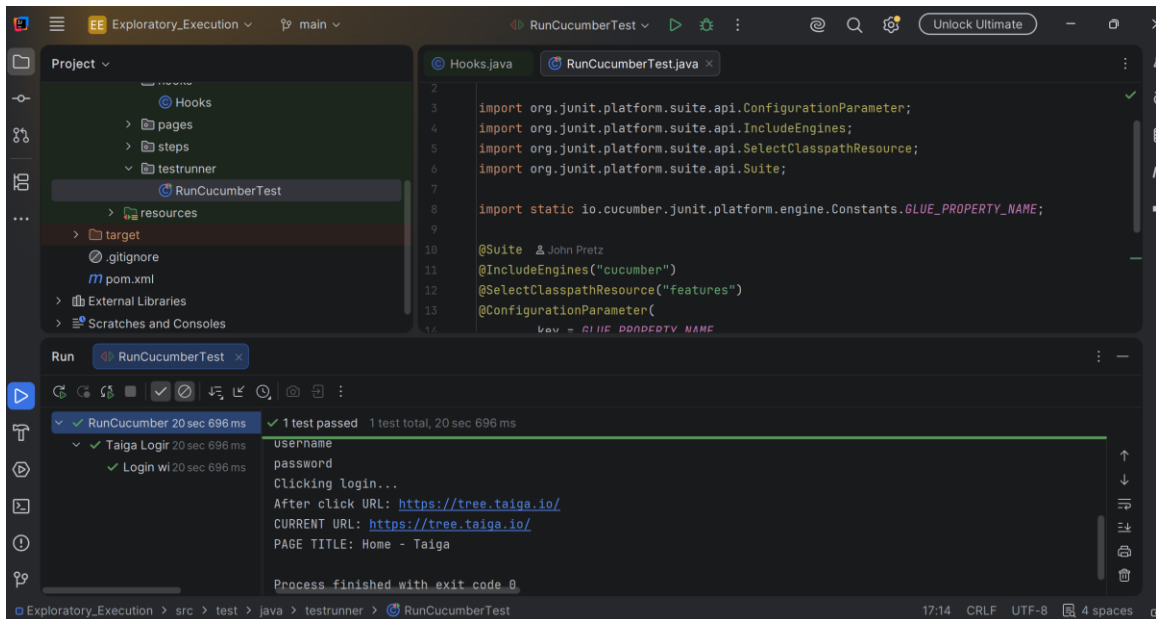
D.4 Summary of Exploratory Testing

The summary table reports the volume of exploratory testing performed and classifies the results by defect severity and observation type.

Metric	Value
Sessions planned/executed	5 / 5
Total scenarios exercised	33
Potential defects (minor)	3 (EX01-09, EX02-02, EX03-05)
Observations (acceptable but notable)	6 (EX01-06, EX02-06, EX03-06, EX05-04, EX05-05, EX05-06)
Cross-cutting usability issue	1 (inconsistent error presentation)
Blocker / Major defects	0

Exploratory testing across the five Taiga areas identified no critical defects. Input validation, uniqueness constraints, the Kanban board, and the role-based permission model all proved robust. Three minor defects were identified: a user story can be marked blocked without a reason being supplied (EX01-09); task creation returns a generic error

rather than a clear field-level message, which is inconsistent with the validation applied to user stories and issues (EX02-02); and a zero-duration sprint is accepted despite the rule requiring the start date to be previous to the finish date (EX03-05). A cross-cutting usability theme is that Taiga presents error information inconsistently, using inline field messages, top red banners, a generic failure notice for tasks, and a 404-page stating that a project "no longer exists" when a member's permissions have been withdrawn. The recommended improvements are to enforce a blocked-reason when a user story is blocked, to standardize error messaging to clear field-level validation, to reject same-day sprints or reword the date rule to read "on or before," and to revise the permission-denied message so that it does not imply deletion.

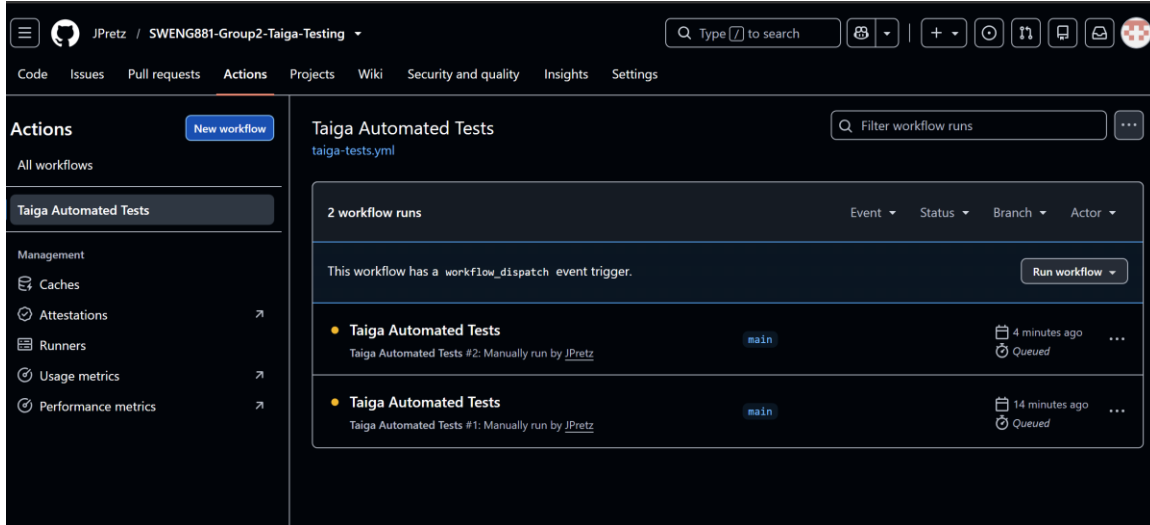


The complete automated JUnit 5 test suite executed twenty-eight (28) total test cases across all feature areas and graph-based authentication scenarios. All twenty-eight tests passed successfully with no execution failures.

D.5 Automated Login Test Using Selenium and Cucumber

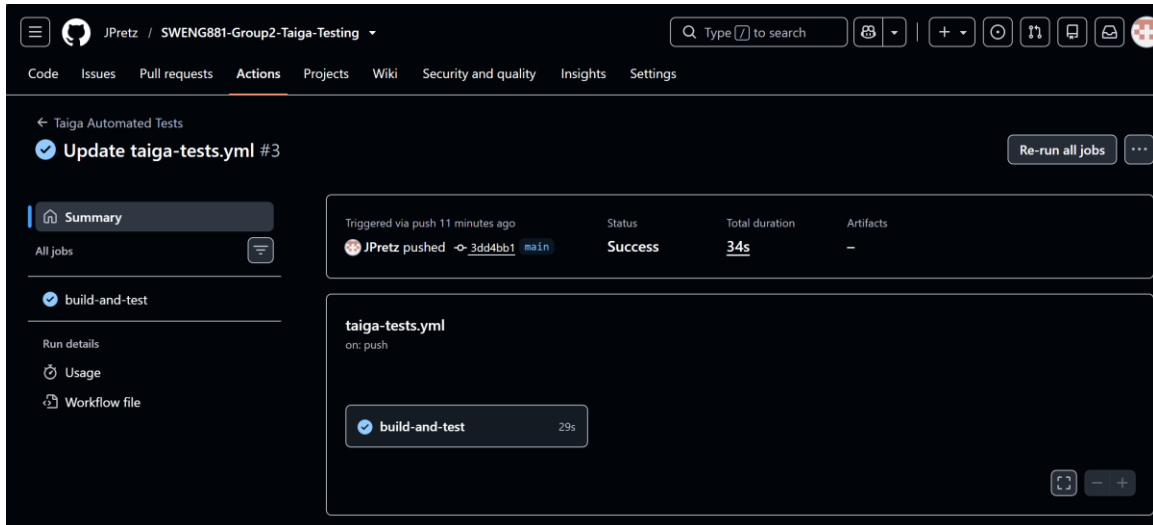
During exploratory testing, an automated login test was developed using Selenium WebDriver, Cucumber, and JUnit 5 to provide a reliable and repeatable way to access the Taiga application. The test automatically launched a web browser, navigated to the Taiga login page, entered valid user credentials, and selected the Login button. Successful authentication was verified by confirming that the application redirected the user from the login page to the Taiga home page (<https://tree.taiga.io/>) and displayed the page title “Home - Taiga.” The test executed successfully, demonstrating that the login functionality operates as expected and providing an authenticated session for subsequent automated testing of user stories, tasks, sprints, and permissions

- The figure below shows the GitHub Actions CI Workflow Queue Status. The GitHub Actions workflow was successfully configured and triggered for the Taiga testing project. The workflow is shown awaiting execution by a GitHub-hosted runner and is designed to automatically execute the complete JUnit 5 automated test suite whenever repository updates occur.



- The Figure below illustrates the successful execution of the GitHub Actions Continuous Integration (CI) workflow implemented for the Taiga testing project. The workflow was automatically triggered by a push event to the GitHub repository and completed successfully in 34 seconds. During execution, GitHub Actions checked out the repository, configured the Java execution environment, and ran the Maven-based automated

tests. The successful completion of the workflow demonstrates the integration of automated testing within a cloud-based CI/CD environment and provides support for continuous verification, regression testing, and software quality assurance activities.



The GitHub Actions Continuous Integration (CI) workflow was successfully triggered by a repository update and completed execution in 34 seconds. The workflow automatically built the Maven project and executed the automated JUnit 5 test suite, demonstrating successful integration of automated testing within a cloud-based CI/CD environment.